



UNIVERSITÀ DI PARMA

DIPARTIMENTO DI SCIENZE MATEMATICHE, FISICHE E INFORMATICHE

Corso di Laurea Triennale in Informatica

Metodologie per Orientamento al Pensiero Computazionale: un Caso di Studio basato sul Problema dello Zaino

*Methodologies for Computational Thinking Orientation: A Case
Study Based on the Knapsack Problem*

CANDIDATO:
Sterling Momo Nguimatsa

RELATORE:
Prof. Vincenzo Bonnici

*Dedico questo lavoro ai miei genitori, per il loro amore incondizionato e il
loro sostegno costante.*

Indice

Introduzione	1
1 Orientamento al Pensiero Computazionale	3
1.1 Cos'è il pensiero computazionale	3
1.2 Perché insegnarlo: obiettivi educativi e formativi	4
1.3 Metodologie didattiche per l'orientamento	4
1.3.1 Programmazione visuale e attività <i>unplugged</i>	5
1.3.2 Learning-by-doing e problem solving	5
1.3.3 Gamification e ambienti interattivi	5
1.3.4 Game-based learning	5
1.4 Il problema dello zaino come strumento didattico	6
1.4.1 Fondamenti teorici	6
1.4.2 Valore educativo del problema dello zaino	9
2 Descrizione e Analisi del Progetto	11
2.1 Obiettivi del Progetto di Tirocinio	11
2.2 Descrizione Generale del Gioco	12
2.3 Strumenti e Tecnologie Utilizzate	13
2.3.1 Tecnologie Front-End	13
2.3.2 Ambiente di Sviluppo e Repository	14
2.4 Analisi dei Requisiti	14
2.4.1 Requisiti Funzionali	15
2.4.2 Requisiti Non Funzionali	16
3 Progettazione tramite Diagrammi UML	17
3.1 Diagrammi dei Casi d'Uso	17
3.1.1 Casi d'Uso Principali	17
3.1.2 Logica Algoritmica	20
3.2 Diagramma delle Attività	21
3.2.1 Attività Principali	21
3.2.2 Diagramma delle Attività dell'Algoritmo CPU	23

3.3	Diagrammi di Flusso delle Attività	25
3.3.1	Flusso Principale dell'Applicazione	25
3.3.2	Flusso dell'Algoritmo Greedy	27
3.3.3	Flusso dell'Algoritmo di Programmazione Dinamica	29
3.3.4	Flusso della Gestione dello Zaino	31
3.4	Diagramma delle classi	33
3.5	Diagrammi di Sequenza	36
3.5.1	Sequenza Principale del Gioco	37
3.5.2	Sequenza Inserimento Oggetto	38
3.5.3	Sequenza dell'Algoritmo Greedy	40
3.5.4	Sequenza dell'Algoritmo di Programmazione Dinamica	42
3.5.5	Sequenza di Fine Partita	43
3.6	Diagrammi dei Componenti	45
3.6.1	Presentation Layer	47
3.6.2	Business Logic Layer	47
3.6.3	Data Layer	47
3.7	Architettura del Progetto	48
4	Implementazione e Validazione	53
4.1	Organizzazione del progetto	53
4.1.1	<code>index.html</code>	53
4.1.2	<code>main.js</code>	54
4.1.3	<code>style.css</code>	55
4.2	Interfaccia utente	55
4.2.1	Struttura generale	55
4.2.2	Aspetti grafici e responsività	58
4.3	Interazioni e funzionalità principali dell'applicazione	58
4.3.1	Inizializzazione e avvio del gioco	59
4.3.2	Selezione dell'algoritmo da sfidare	59
4.3.3	Selezione e interazione con gli oggetti	60
4.3.4	Gestione dell'inserimento e rimozione nello zaino	60
4.3.5	Gestione dei turni e sincronizzazione	62
4.3.6	Esecuzione dell'algoritmo selezionato	63
4.3.7	Conclusione della partita e valutazione finale	64
4.3.8	Comunicazione e feedback all'utente	65
4.4	Algoritmi Implementati	66
4.4.1	Algoritmo Greedy	66
4.4.2	Algoritmo di Programmazione Dinamica	67
4.4.3	Confronto tra gli Approcci	69
4.5	Test e Validazione	69
4.5.1	Strategia di test	70

4.5.2	Analisi dei risultati	72
5	Risultati	75
5.1	Nuova partita	76
5.2	Sfida l'algoritmo	77
5.2.1	Gestione del peso	83
5.3	Fine della partita e confronto dei risultati	89
5.3.1	Fine della partita: Utente vs Greedy	89
5.3.2	Caso dell'algoritmo di Programmazione Dinamica	90
	Bibliografia	95

Elenco delle figure

3.1	Diagramma dei casi d'uso principale che presenta le interazioni tra utente e applicazione	18
3.2	Diagramma UML che modella le strategie algoritmiche per la risoluzione del problema dello zaino	20
3.3	Diagramma delle Attività del Gioco sullo Zaino	22
3.4	Diagramma delle Attività: Algoritmo CPU (Greedy / DP)	24
3.5	Flusso Principale dell'Applicazione	26
3.6	Flusso Algoritmo Greedy	28
3.7	Flusso Programmazione Dinamica	30
3.8	Flusso Gestione Zaino	32
3.9	Classe Principale dell'Applicazione	34
3.10	Sequenza Principale dell'Applicazione	37
3.11	Sequenza di Inserimento Oggetto da Parte Utente	39
3.12	Sequenza di Esecuzione dell'Algoritmo di Greedy	41
3.13	Sequenza di Esecuzione dell'Algoritmo di Programmazione Dinamica	42
3.14	Sequenza di Fine Partia	44
3.15	Vista a livelli dell'architettura dell'applicativo	46
3.16	Architettura del Sistema	49
5.1	Avvio della pagina HTML	76
5.2	Avvio di una nuova partita	77
5.3	Area di controllo	78
5.4	Sezione del giocatore (algoritmo)	79
5.5	Primo turno dell'utente: selezione dell'oggetto	80
5.6	Primo turno dell'utente: inserimento dell'oggetto	81
5.7	Prima mossa dell'algoritmo	82
5.8	Situazione di peso superato: caso 1	83
5.9	Cambio di item (nuova selezione)	84
5.10	Nuovo inserimento riuscito	85
5.11	Situazione di peso superato: caso 2	86
5.12	Selezione e rimozione di un oggetto dallo zaino	87

5.13	Nuovo inserimento riuscito	88
5.14	Fine della partita	89
5.15	Fine della partita: vittoria dell'algoritmo	90
5.16	Fine della partita: vittoria dell'utente	91
5.17	Fine della partita: pareggio	92

Elenco degli algoritmi

1	Greedy per lo zaino 0/1	7
2	Greedy per lo zaino frazionario	8
3	Zaino 0/1 con programmazione dinamica	8
4	Branch and Bound per lo zaino 0/1	9

Introduzione

Negli ultimi anni, il pensiero computazionale è diventato un tema centrale nel panorama educativo, poiché rappresenta una competenza trasversale fondamentale per affrontare problemi, pianificare soluzioni e ragionare in modo strutturato. Sviluppare questa abilità non significa semplicemente imparare a programmare, ma piuttosto acquisire un modo di pensare utile in molti contesti della vita quotidiana e professionale. Per questo motivo, sempre più iniziative scolastiche e progetti didattici si stanno orientando verso metodologie innovative che aiutino gli studenti a familiarizzare con i principi dell'informatica in modo intuitivo e coinvolgente.

Tra le varie strategie educative, l'utilizzo del gioco come strumento di apprendimento ha dimostrato una grande efficacia. Ambienti interattivi, dinamiche ludiche e sfide digitali stimolano curiosità, motivazione e partecipazione attiva, rendendo l'apprendimento un'esperienza concreta e significativa. All'interno di questo contesto, il problema dello zaino (*knapsack problem*) si presta particolarmente bene come attività didattica. Si tratta di un classico problema algoritmico che, pur nella sua semplicità apparente, richiede logica, capacità di pianificazione e valutazione delle scelte. Il suo valore formativo è doppio: da un lato, permette di introdurre concetti chiave dell'ottimizzazione e degli algoritmi; dall'altro, offre uno scenario pratico e accessibile per esercitarsi nel *problem solving*.

Questa tesi nasce con l'intento di valorizzare il problema dello zaino come strumento per l'orientamento al pensiero computazionale. Per farlo, è stato realizzato un gioco educativo in cui l'utente interagisce con oggetti virtuali, simula strategie risolutive e compete con un algoritmo, all'interno di un'interfaccia semplice e intuitiva. Il progetto si pone l'obiettivo di coniugare rigore informatico e facilità d'uso, rendendo l'esperienza adatta sia a studenti che a chi si avvicina per la prima volta all'informatica.

Il lavoro è strutturato in quattro capitoli principali, ciascuno dei quali affronta in modo progressivo e coerente i diversi aspetti del progetto.

In apertura, nel primo capitolo viene introdotto il concetto di pensiero computazionale, sottolineandone l'importanza nei contesti educativi e pre-

sentando alcune delle metodologie più diffuse per svilupparlo, come la programmazione visuale, il learning-by-doing e l'apprendimento basato sul gioco. In seguito, si approfondisce il valore didattico del *problema dello zaino*, analizzandone le varianti principali e le strategie algoritmiche più comuni, come l'approccio greedy e la programmazione dinamica.

Il secondo capitolo è dedicato alla descrizione e all'analisi del progetto di tirocinio. Si delineano gli obiettivi formativi dell'applicativo, le meccaniche interattive proposte e le funzionalità chiave. Vengono inoltre definiti i requisiti funzionali e non funzionali del sistema, con un focus sulla fruibilità da parte di utenti senza conoscenze pregresse di programmazione.

Nel terzo capitolo si passa alla fase di progettazione, presentando l'architettura del sistema attraverso i principali diagrammi UML: casi d'uso, attività, classi, sequenza, componenti e flusso logico. L'obiettivo è fornire una rappresentazione strutturata dell'organizzazione interna dell'applicativo, evidenziando i ruoli delle diverse componenti e le loro interazioni.

Infine, l'ultimo capitolo si concentra sull'implementazione dell'applicazione e sulla sua validazione con presentazione del risultato finale. Vengono descritte le principali scelte tecniche, la struttura dei file e le funzionalità più rilevanti del codice. Infine, viene illustrato il processo di test e validazione, sia sul piano funzionale che su quello educativo, con una riflessione sulle potenzialità didattiche dello strumento e sui possibili sviluppi futuri.

Attraverso questo percorso, si intende mostrare come anche un classico problema informatico possa trasformarsi in una risorsa formativa efficace, se accompagnato da scelte progettuali attente e da una visione didattica orientata all'esperienza dell'utente.

Capitolo 1

Orientamento al Pensiero Computazionale

1.1 Cos'è il pensiero computazionale

Il pensiero computazionale (*computational thinking*) è un insieme di processi mentali utilizzati per risolvere problemi complessi in modo sistematico e strutturato, seguendo modalità tipiche dell'informatica. Non si tratta di saper programmare, ma di acquisire la capacità di analizzare un problema, scomporlo in parti più semplici, riconoscere schemi ricorrenti, astrarre informazioni rilevanti e ideare una procedura o algoritmo per affrontarlo.

Questo tipo di pensiero si basa su concetti fondamentali come:

- **Decomposizione**, ovvero la suddivisione di un problema in sotto-problemi più gestibili;
- **Astrazione**, cioè l'identificazione degli elementi essenziali eliminando i dettagli irrilevanti;
- **Riconoscimento di schemi** (*pattern recognition*), utile per individuare somiglianze tra problemi differenti;
- **Algoritmi**, ovvero la definizione di una sequenza precisa di passi per risolvere un problema.

Il pensiero computazionale è quindi una competenza trasversale che può essere applicata in numerosi contesti, non solo nell'informatica, ma anche nella matematica, nelle scienze, nella logica e nella vita quotidiana. È uno strumento di problem solving che aiuta a sviluppare autonomia, pensiero critico e capacità di ragionamento.

1.2 Perché insegnarlo: obiettivi educativi e formativi

Negli ultimi anni il pensiero computazionale è stato riconosciuto come una delle competenze chiave per l'educazione del XXI secolo. La crescente pervasività delle tecnologie digitali nella vita quotidiana rende sempre più urgente offrire agli studenti gli strumenti per comprendere, analizzare e utilizzare tali tecnologie in modo consapevole e attivo.

L'insegnamento del pensiero computazionale a scuola ha molteplici finalità:

- **Formare cittadini digitali consapevoli**, capaci di comprendere come funzionano gli strumenti che utilizzano quotidianamente;
- **Sviluppare il problem solving e la logica**, competenze essenziali non solo in ambito STEM (Science, Technology, Engineering, Mathematics), ma anche in contesti interdisciplinari;
- **Favorire la creatività e la progettualità**, attraverso attività che stimolano l'ideazione e la costruzione di soluzioni originali;
- **Promuovere l'inclusione e la collaborazione**, mediante attività che valorizzano il lavoro di gruppo e l'apprendimento tra pari.

L'insegnamento del pensiero computazionale non si limita dunque all'introduzione precoce della programmazione, ma costituisce un'opportunità per innovare la didattica, rendendola più attiva, laboratoriale e centrata sullo studente.

1.3 Metodologie didattiche per l'orientamento

Per introdurre efficacemente il pensiero computazionale nella scuola, è fondamentale adottare metodologie didattiche che stimolino la partecipazione attiva, la sperimentazione e l'apprendimento attraverso l'esperienza. L'obiettivo è avvicinare gli studenti a concetti astratti mediante attività concrete, coinvolgenti e accessibili, che facilitino la comprensione e favoriscano l'interiorizzazione delle strategie di problem solving.

Le metodologie più diffuse ed efficaci per l'orientamento al pensiero computazionale includono approcci visuali e unplugged, attività laboratoriali pratiche, giochi didattici e tecniche di gamification. Di seguito ne vengono descritte le principali.

1.3.1 Programmazione visuale e attività *unplugged*

La programmazione visuale consente agli studenti di costruire algoritmi utilizzando blocchi grafici e istruzioni visive, senza dover scrivere codice testuale. Ambienti come SCRATCH, BLOCKLY o CODE.ORG permettono di sviluppare competenze computazionali in modo intuitivo e creativo, anche nei primi anni della scuola primaria.

Le attività *unplugged*, invece, introducono i concetti dell'informatica senza l'uso del computer, attraverso giochi di ruolo, puzzle cartacei e attività motorie. Questo approccio rende il pensiero computazionale accessibile a tutti, riducendo le barriere tecnologiche e promuovendo una comprensione più concreta dei meccanismi alla base degli algoritmi e dei processi informatici.

1.3.2 Learning-by-doing e problem solving

Il metodo del *learning-by-doing*, ovvero dell'apprendere facendo, è alla base delle attività che pongono gli studenti al centro del processo educativo. L'idea è che la conoscenza si costruisca attraverso l'esperienza diretta, il confronto con problemi reali e la riflessione sulle proprie azioni.

All'interno di questo approccio, le attività di problem solving svolgono un ruolo cruciale. Gli studenti sono chiamati a risolvere problemi aperti, a esplorare soluzioni alternative e a sperimentare strategie algoritmiche, sviluppando così non solo competenze logiche ma anche autonomia e pensiero critico.

1.3.3 Gamification e ambienti interattivi

La *gamification* consiste nell'integrare elementi tipici del gioco (punti, livelli, premi, classifiche) in contesti non ludici, come l'educazione. Questo approccio mira a rendere l'apprendimento più coinvolgente e motivante, mantenendo alta l'attenzione e incentivando la partecipazione.

In combinazione con ambienti interattivi digitali, la gamification permette di creare percorsi personalizzati di apprendimento, stimolare la perseveranza e favorire una comprensione più profonda dei concetti computazionali. Strumenti come simulazioni, laboratori virtuali e giochi interattivi forniscono feedback immediati e una didattica centrata sullo studente.

1.3.4 Game-based learning

L'apprendimento basato sul gioco (*game-based learning*) utilizza il gioco come strumento per veicolare contenuti educativi. A differenza della gamification,

qui il gioco non è solo un supporto motivazionale, ma costituisce l'ambiente principale attraverso cui avviene l'apprendimento.

Attraverso giochi educativi digitali o analogici, gli studenti affrontano sfide strutturate che richiedono l'applicazione di strategie computazionali. Questi contesti favoriscono l'attenzione, la motivazione e la partecipazione attiva, stimolando al contempo la riflessione metacognitiva sui processi decisionali e algoritmici.

1.4 Il problema dello zaino come strumento didattico

1.4.1 Fondamenti teorici

Nel contesto dell'insegnamento del pensiero computazionale, il problema dello zaino (in inglese *Knapsack Problem*) rappresenta un esempio efficace e stimolante per introdurre gli studenti alla logica algoritmica e all'ottimizzazione combinatoria.

Nella sua formulazione classica, si considera un insieme di oggetti, ciascuno caratterizzato da un valore e un peso, e un contenitore (lo zaino) con capacità massima limitata. L'obiettivo è selezionare una combinazione di oggetti tale da massimizzare il valore totale, senza superare la capacità dello zaino:

- **Input:** un insieme di n oggetti, ciascuno con valore v_i e peso w_i , e una capacità massima W .
- **Output:** una selezione binaria di oggetti ($x_i \in \{0, 1\}$) tale che:
$$\sum w_i x_i \leq W \quad \text{e} \quad \sum v_i x_i \text{ sia massimizzata.}$$

Grazie alla sua semplicità di formulazione e alla profondità delle possibili soluzioni algoritmiche, il problema si presta sia ad attività introduttive sia a percorsi più avanzati, adattabili a diversi livelli scolastici.

Varianti principali del *Knapsack Problem*

Esistono numerose varianti del problema, tra cui:

- **Problema dello zaino 0/1:** ciascun oggetto può essere selezionato al massimo una volta. È la versione più studiata e comunemente utilizzata nella didattica.

CAPITOLO 1. ORIENTAMENTO AL PENSIERO COMPUTAZIONALE

- **Zaino frazionario:** è consentito inserire frazioni di oggetti. In questo caso, un algoritmo greedy garantisce la soluzione ottimale.
- **Zaino multidimensionale:** ogni oggetto ha più attributi (es. peso, volume, tempo), e lo zaino è soggetto a vincoli multipli.
- **Zaino multiplo:** si dispone di più zaini da riempire con lo stesso insieme di oggetti.
- **Zaino con dipendenze:** la selezione di un oggetto può dipendere dalla presenza o assenza di altri oggetti.

Il problema dello zaino 0/1 è *NP-completo*, ovvero non è noto alcun algoritmo efficiente in grado di risolvere tutte le istanze in tempo polinomiale. Tuttavia, esistono metodi risolutivi esatti e approssimati di grande interesse didattico.

Algoritmi risolutivi

Algoritmo greedy (avido): L'idea di base è ordinare gli oggetti in base a un criterio (ad esempio, il rapporto valore/peso $r_i = v_i/w_i$) e inserirli nello zaino finché c'è spazio disponibile. Questo approccio è intuitivo, semplice da implementare e utilizza una strategia avida: prende decisioni localmente ottimali nella speranza di ottenere una soluzione globalmente soddisfacente.

Algoritmo 1 Greedy per lo zaino 0/1

- 1: Ordina gli oggetti decrescentemente per v_i/w_i
 - 2: $W \leftarrow$ capacità residua dello zaino
 - 3: **for** ogni oggetto i in ordine **do**
 - 4: **if** $w_i \leq W$ **then**
 - 5: inserisci tutto i nello zaino
 - 6: $W \leftarrow W - w_i$
 - 7: **else**
 - 8: salta l'oggetto i
-

Algoritmo 2 Greedy per lo zaino frazionario

```

1: Ordina gli oggetti decrescentemente per  $v_i/w_i$ 
2:  $W \leftarrow$  capacità residua dello zaino
3: for ogni oggetto  $i$  in ordine do
4:   if  $w_i \leq W$  then
5:     inserisci tutto  $i$  nello zaino
6:      $W \leftarrow W - w_i$ 
7:   else
8:     inserisci frazione  $W/w_i$  dell'oggetto  $i$ 
9:   break

```

Per la versione 0/1, l'approccio greedy non garantisce una soluzione ottimale, ma è risolutivo nel caso frazionario, dove l'ottimalità è assicurata.

Programmazione dinamica: Si tratta di un approccio esatto per la variante 0/1, particolarmente efficace quando W non è troppo grande. Viene costruita una tabella $DP[i][w]$ in cui i rappresenta il numero di oggetti considerati e w la capacità disponibile dello zaino. La programmazione dinamica suddivide il problema in sottoproblemi più piccoli, risolvendoli ricorsivamente e memorizzando i risultati intermedi per evitare calcoli ridondanti.

Algoritmo 3 Zaino 0/1 con programmazione dinamica

```

1: Inizializza  $DP[0..n][0..W] \leftarrow 0$ 
2: for  $i = 1$  to  $n$  do
3:   for  $w = 0$  to  $W$  do
4:     if  $w_i \leq w$  then
5:        $DP[i][w] \leftarrow \max(DP[i-1][w], v_i + DP[i-1][w - w_i])$ 
6:     else
7:        $DP[i][w] \leftarrow DP[i-1][w]$ 
8: return  $DP[n][W]$ 

```

Grazie alla sua strategia, trova sempre la soluzione ottimale, benché la complessità computazionale sia elevata (sia in tempo che in spazio), soprattutto per valori elevati di n e W .

Altri metodi: Oltre ai metodi sopra descritti, esistono approcci più avanzati e flessibili:

Branch and Bound: Utilizza una struttura ad albero per esplorare solo le soluzioni promettenti, potando rami che non possono migliorare la solu-

zione corrente con un costo computazionale nel caso peggiore esponenziale, ma più efficiente in pratica rispetto alla ricerca esaustiva.

Algoritmo 4 Branch and Bound per lo zaino 0/1

```
1: Ordina gli oggetti decrescentemente per  $v_i/w_i$ 
2: Inizializza una coda di nodi (stato: livello, valore, peso, bound)
3: Inserisci il nodo radice (livello 0, valore 0, peso 0, bound calcolato)
4:  $maxValue \leftarrow 0$ 
5: while la coda non è vuota do
6:   Estrai il nodo con bound più alto
7:   if il bound del nodo  $\leq maxValue$  then
8:     continua
9:   if livello == n then
10:    continua
11:   // Prova a includere l'oggetto successivo
12:   if peso +  $w_{livello} \leq W$  then
13:     Crea nuovo nodo con oggetto incluso
14:     if valore nuovo >  $maxValue$  then
15:        $maxValue \leftarrow$  valore nuovo
16:     Calcola bound e inserisci in coda
17:   // Prova a escludere l'oggetto successivo
18:   Crea nuovo nodo senza oggetto incluso
19:   Calcola bound e inserisci in coda
20: return  $maxValue$ 
```

Programmazione Lineare Intera (ILP): Modella il problema come un sistema di vincoli lineari con variabili intere, risolubile con solutori ottimizzati (es. CPLEX, Gurobi). È generalizzabile a problemi più complessi con vincoli addizionali.

1.4.2 Valore educativo del problema dello zaino

Dal punto di vista didattico, il problema dello zaino offre numerose opportunità formative. Gli studenti sono chiamati a prendere decisioni strategiche in condizioni di vincoli e risorse limitate, sviluppando così competenze legate alla pianificazione, all'analisi costi-benefici e al problem solving.

Inoltre, la sua natura concreta e facilmente rappresentabile consente di progettare attività laboratoriali coinvolgenti, in cui oggetti fisici o visuali rendono tangibile il processo decisionale. Questo favorisce non solo l'apprendimento dei concetti algoritmici, ma anche lo sviluppo della consapevolezza delle proprie strategie cognitive.

CAPITOLO 1. ORIENTAMENTO AL PENSIERO COMPUTAZIONALE

Utilizzato in ambienti interattivi o ludici, come nel caso del progetto oggetto di questa tesi, il problema dello zaino consente di confrontare in tempo reale le scelte dell'utente con quelle ottimizzate da un algoritmo, stimolando la riflessione e la comprensione del funzionamento dei metodi computazionali.

Capitolo 2

Descrizione e Analisi del Progetto

Il presente capitolo descrive il progetto di tirocinio universitario, realizzato nell'ambito dell'iniziativa *Sfida l'Algoritmo*, promossa dall'Università degli Studi di Parma – Dipartimento di Scienze Matematiche, Fisiche e Informatiche. Il progetto si inserisce nel contesto della didattica informatica e dell'orientamento al pensiero computazionale, con particolare attenzione all'apprendimento degli algoritmi attraverso metodologie innovative basate sull'interazione e sulla visualizzazione.

2.1 Obiettivi del Progetto di Tirocinio

Il progetto ha come obiettivo la realizzazione di un'applicazione interattiva per l'apprendimento del *problema dello zaino* (*Knapsack Problem*), utilizzando meccaniche proprie del Game-Based Learning (GBL) e della Gamification per favorire la partecipazione attiva degli studenti.

L'iniziativa mira a introdurre concetti algoritmici complessi attraverso un'interfaccia intuitiva e una rappresentazione visiva coinvolgente, permettendo agli utenti di sviluppare competenze computazionali in modo pratico, esplorando strategie risolutive in un contesto ludico.

Gli obiettivi principali del progetto sono:

- **Introdurre il problema dello zaino** tramite una rappresentazione grafica interattiva che consenta agli utenti di visualizzare gli oggetti come elementi bidimensionali e comprenderne le caratteristiche principali: peso, valore e capacità limitata dello zaino.
- **Sviluppare un'interfaccia intuitiva e coinvolgente**, che permetta di interagire con gli oggetti mediante selezione e pulsanti, simulando

decisioni reali di ottimizzazione in un ambiente visuale orientato al gioco.

- **Integrare un algoritmo di risoluzione** basato su diversi algoritmi risolutivi (Greedy, Programmazione Dinamica) all'interno del gioco, per offrire un confronto diretto tra le scelte dell'utente e quelle compiute dall'algoritmo.
- **Stimolare l'apprendimento attivo** dei concetti algoritmici e la riflessione sugli aspetti computazionali attraverso la sperimentazione diretta, il ragionamento sulle soluzioni ottenute e la riflessione sui limiti delle strategie implementate.
- **Sviluppare competenze tecniche e pratiche** nello sviluppo web, mediante l'utilizzo di HTML, CSS e JavaScript, organizzando il codice secondo un'architettura modulare ispirata al pattern Model-View-Controller (MVC).

2.2 Descrizione Generale del Gioco

L'applicazione sviluppata consiste in un *gioco interattivo a turni* che simula il problema dello zaino con vincoli di peso (spazio). L'utente ha a disposizione una selezione di oggetti, ciascuno caratterizzato da peso e valore, rappresentati visivamente come barre colorate con icone (coin) proporzionali al valore. L'utente ha a disposizione una selezione di oggetti, ciascuno caratterizzato da peso e valore, rappresentati visivamente come barre colorate con icone (coin) proporzionali al valore e uno zaino con capacità (peso) limitata da riempire.

Il gioco si articola in due modalità principali:

- **Modalità Utente:** l'utente seleziona e inserisce gli oggetti all'interno dello zaino, cercando di massimizzare il valore totale (numero di coin) senza superare la capacità spaziale disponibile. Durante il gioco, l'utente riceve feedback visivi sulla validità dell'inserimento e sull'aggiornamento del valore totale ottenuto.
- **Modalità Algoritmo:** viene eseguita automaticamente una strategia algoritmica (es. Greedy), che simula una possibile soluzione ottima o approssimata. Il confronto tra la soluzione dell'utente e quella proposta dall'algoritmo consente di analizzare l'efficacia delle scelte compiute.

La griglia dello zaino è rappresentata come una matrice di celle (es. 10×2) in cui vengono inseriti gli oggetti, tenendo conto dell'orientamento e delle

dimensioni. Gli oggetti non inseribili vengono scartati, e l'utente può decidere di rimuoverli per fare spazio a nuovi inserimenti.

Al termine della partita, il sistema calcola e visualizza:

- il valore totale ottenuto dall'utente;
- il valore ottenuto dall'algoritmo;
- la quantità di peso occupato da ciascuno;
- una valutazione comparativa con classifica o messaggi motivazionali.

Questo approccio ludico favorisce l'introduzione del problema dello zaino, l'apprendimento attivo, la sperimentazione e l'osservazione dei limiti computazionali delle strategie implementate.

2.3 Strumenti e Tecnologie Utilizzate

Lo sviluppo del progetto è stato condotto utilizzando esclusivamente tecnologie web open source, accessibili e compatibili con un'esecuzione locale su qualsiasi browser moderno. L'obiettivo è stato quello di creare un ambiente semplice, ma didatticamente efficace, che potesse essere facilmente distribuito e utilizzato anche in contesti scolastici con risorse limitate.

2.3.1 Tecnologie Front-End

- **HTML5 (HyperText Markup Language)** è stato impiegato per la struttura del documento e la definizione dei principali elementi dell'interfaccia utente, come il contenitore degli oggetti disponibili, lo zaino e i pannelli informativi. La semplicità della struttura HTML ha favorito l'organizzazione modulare dei componenti visivi e ha facilitato l'accessibilità del contenuto.
- **CSS3 (Cascading Style Sheets)** è stato utilizzato per la definizione dello stile grafico dell'applicazione. In particolare, le regole CSS hanno permesso la rappresentazione degli oggetti come barre colorate proporzionali al peso, contenenti icone a forma di moneta (coin) proporzionali al valore. Le griglie di disposizione, le transizioni visive e l'allineamento degli elementi sono stati curati per rendere l'esperienza utente intuitiva e coerente.

- **JavaScript (ES6)** è stato il linguaggio di programmazione principale per la logica dell'applicazione. Le funzionalità comprendono la gestione della selezione degli oggetti, l'inserimento nello zaino, il calcolo del valore e del peso totale, la visualizzazione in tempo reale delle modifiche, e l'esecuzione automatica della strategia scelta per il confronto algoritmico. La programmazione ad oggetti e l'uso di classi ha permesso di strutturare il codice in modo chiaro e riusabile.

2.3.2 Ambiente di Sviluppo e Repository

L'ambiente di sviluppo principale è stato **Visual Studio Code**, editor moderno e flessibile, che ha permesso una gestione efficiente dei file, del debugging e dell'organizzazione del progetto. Sono stati utilizzati estensioni come Live Server per il testing immediato delle modifiche e strumenti di linting per il controllo della qualità del codice JavaScript.

Durante lo sviluppo, è stato utilizzato **Git** come sistema di controllo versione, in combinazione con la piattaforma **GitHub** per il salvataggio remoto e la collaborazione. Il repository contiene l'intero codice sorgente del progetto, comprensivo di file HTML, CSS, JavaScript e documentazione ausiliaria.

Il codice è disponibile pubblicamente al seguente indirizzo:

[https://github.com/vbonnici/edu-algos/tree/
36a811ce903965f2fec5a9248a84dd27d44f343f/Zaino_Game](https://github.com/vbonnici/edu-algos/tree/36a811ce903965f2fec5a9248a84dd27d44f343f/Zaino_Game)

Questa scelta ha permesso di mantenere una cronologia chiara delle modifiche, facilitare il backup, e rendere il progetto accessibile ad altri studenti, docenti o collaboratori interessati a riutilizzare o estendere l'applicazione.

Per quanto riguarda la portabilità, l'intero progetto è **client-side**, non richiede installazioni aggiuntive né dipendenze esterne, ed è compatibile con i principali browser moderni (Chrome, Firefox, Edge, Safari). È possibile eseguirlo localmente aprendo il file `index.html`, rendendo l'applicazione particolarmente adatta a contesti educativi.

2.4 Analisi dei Requisiti

L'analisi dei requisiti rappresenta un passaggio fondamentale nella progettazione del sistema, in quanto consente di definire in modo chiaro e strutturato le funzionalità che l'applicazione deve offrire, nonché le qualità attese dal punto di vista dell'utente e delle prestazioni. I requisiti sono stati raccolti

tenendo conto degli obiettivi didattici del progetto, dell'ambiente di esecuzione previsto (browser web), e delle aspettative degli utenti target (studenti e docenti).

2.4.1 Requisiti Funzionali

I requisiti funzionali descrivono le azioni che il sistema deve essere in grado di eseguire. Nel contesto del gioco educativo sul problema dello zaino, i principali requisiti funzionali sono i seguenti:

- **Visualizzazione degli oggetti disponibili:** Il sistema deve mostrare un insieme di oggetti, ciascuno caratterizzato da peso e valore, rappresentati graficamente come barre orizzontali proporzionali.
- **Scelta dell'algoritmo:** Offrire all'utente la possibilità di selezionare l'algoritmo da sfidare (dropdown con opzioni Greedy/Programmazione Dinamica).
- **Selezione degli oggetti:** L'utente deve poter selezionare uno degli oggetti disponibili cliccando sull'elemento corrispondente.
- **Inserimento manuale nello zaino:** L'utente deve poter inserire l'oggetto selezionato nello zaino premendo un apposito pulsante **Inserisci Item**.
- **Gestione dei vincoli:** Impedire l'inserimento di oggetti se il peso eccede la capacità residua dello zaino.
- **Rimozione dallo zaino:** L'utente deve poter rimuovere un oggetto già inserito selezionandolo e premendo il pulsante **Rimuovi Item**.
- **Aggiornamento dello stato:** Dopo ogni inserimento o rimozione, il sistema deve aggiornare e visualizzare in tempo reale il peso totale, il valore complessivo e la capacità residua dello zaino.
- **Divisione degli oggetti su righe:** Gli oggetti inseriti nello zaino devono occupare una griglia visiva (10x2), con disposizione automatica da sinistra a destra e su due righe in caso di overflow orizzontale.
- **Interazione utente-algoritmo in modo automatico:** il sistema deve abilitare l'algoritmo (Greedy o Programmazione Dinamica) a rispondere automaticamente dopo ogni mossa dell'utente, permettendolo di effettuare una sua mossa.

- **Confronto tra soluzioni:** Alla fine della partita, il sistema deve visualizzare un confronto tra il punteggio ottenuto dall'utente e quello calcolato dall'algoritmo, poi dichiarare il vincitore basandosi sul valore accumulato e, in caso di pareggio, sul peso utilizzato.
- **Riavvio della partita:** Deve essere presente un pulsante per resettare il gioco, svuotando lo zaino e generando un nuovo insieme di oggetti casuali.

2.4.2 Requisiti Non Funzionali

I requisiti non funzionali descrivono le qualità generali che il sistema deve possedere, indipendentemente dalle sue specifiche funzionalità operative. Essi comprendono aspetti legati all'usabilità, alla portabilità e alla manutenibilità:

- **Usabilità:** L'interfaccia deve essere semplice e intuitiva, pensata per un pubblico di studenti anche non esperti di informatica. I pulsanti devono essere chiaramente visibili e le azioni facilmente comprensibili.
- **Reattività:** Tutte le interazioni devono avvenire in tempo reale, senza ritardi percepibili nell'inserimento, rimozione o aggiornamento dei dati.
- **Affidabilità:** Gestione robusta degli errori (es. peso massimo superato).
- **Compatibilità:** L'applicazione deve funzionare correttamente nei principali browser moderni (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari), senza richiedere installazioni esterne.
- **Portabilità:** Il progetto deve essere completamente eseguibile in locale, semplicemente aprendo il file `index.html`, senza bisogno di server web o ambienti di runtime particolari.
- **Accessibilità didattica:** Il contenuto visivo deve essere facilmente interpretabile da parte di studenti delle scuole secondarie, con un uso limitato di testo e una grafica simbolica efficace.
- **Manutenibilità del codice:** Il codice sorgente deve essere suddiviso in moduli logici, strutturato secondo un'architettura leggibile (MVC) e documentato in modo essenziale, per consentirne una facile modifica o estensione futura.

Capitolo 3

Progettazione tramite Diagrammi UML

La fase di progettazione rappresenta un passaggio cruciale per formalizzare il comportamento del sistema e garantire una corrispondenza diretta tra i requisiti e l'implementazione. Attraverso i diagrammi UML, è possibile visualizzare in modo strutturato le interazioni tra utente e sistema, i flussi operativi principali e alternativi, la logica algoritmica alla base delle scelte progettuali, l'architettura dei componenti software. Questo approccio metodologico permette di validare l'aderenza ai requisiti funzionali e non funzionali, identificare potenziali criticità prima dell'implementazione e documentare le scelte progettuali per manutenzione futura.

3.1 Diagrammi dei Casi d'Uso

Il diagramma dei casi d'uso modella le principali funzionalità offerte dall'applicazione, focalizzandosi sulle interazioni tra l'utente e il sistema. Questo schema fornisce una panoramica dei casi d'uso del gioco, facendo astrazione dalla complessità implementativa.

3.1.1 Casi d'Uso Principali

Il diagramma dei casi d'uso principale (Figura 3.1) illustra le interazioni fondamentali tra l'utente e il sistema, con un focus sulle funzionalità chiave dell'applicazione come l'avvio e configurazione della partita, la gestione interattiva degli oggetti nello zaino, il confronto tra le strategie umana e algoritmica.

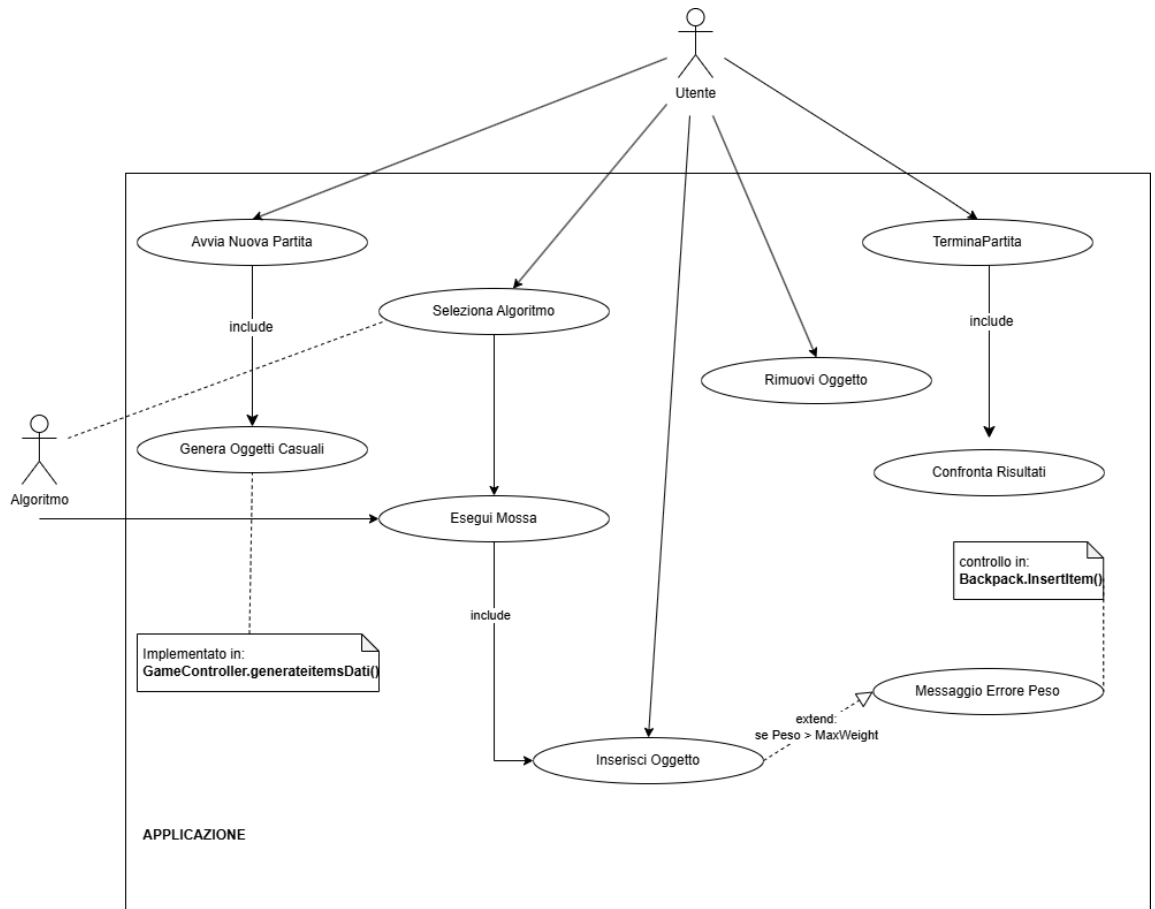


Figura 3.1: Diagramma dei casi d’uso principale che presenta le interazioni tra utente e applicazione

Attori Principali

- **Utente**: Giocatore che interagisce con l’interfaccia.
- **Algoritmo (Greedy/DP)**: Risponde automaticamente eseguendo una mossa.

Casi d’Uso Principali

- **Avvia Nuova Partita**: L’utente inizia una nuova sessione di gioco con parametri predefiniti (peso massimo) e oggetti casuali;
- **Seleziona Algoritmo**: Scelta tra Greedy e Programmazione Dinamica;

- **Inserisci/Rimuovi Oggetto:** L'utente seleziona oggetto, poi lo inserisce/rimuove item dallo zaino secondo i vincoli di peso;
- **Esegui Mossa:** L'algoritmo selezionato risponde automaticamente alla mossa dell'utente inserendo a suo turno un oggetto nello zaino a secondo della sua strategia;
- **Termina Partita:** chiude la partita e fa il confronto dei punteggi.

Relazioni

- **Linee continue:** collegano gli attori ai casi d'uso principali, indicando chi esegue ogni azione.
- **include:** Generazione oggetti è parte obbligatoria dell'avvio
- **extend:** Mostra errore solo se il peso supera il limite (controllato in `Backpack.insertItem()`)
- **le note:** evidenziano i metodi core.

3.1.2 Logica Algoritmica

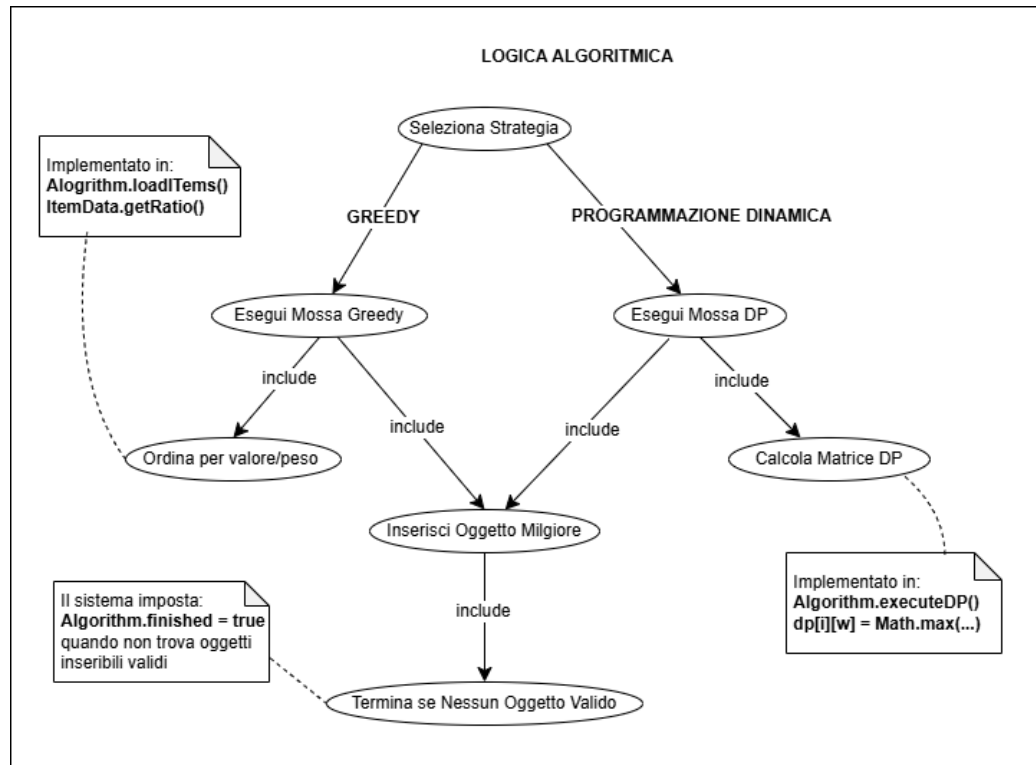


Figura 3.2: Diagramma UML che modella le strategie algoritmiche per la risoluzione del problema dello zaino

Il diagramma descrive in dettaglio le due strategie algoritmiche implementate nel sistema:

Approccio Greedy

- Seleziona gli oggetti in base al rapporto valore/peso decrescente;
- Inserisce progressivamente l'oggetto più vantaggioso che rispetta i vincoli.

Programmazione Dinamica

- Calcola la soluzione ottimale attraverso una matrice di stato;
- Ricostruisce la selezione finale dagli oggetti disponibili.

Casi d'uso

- Utente seleziona "Greedy";
- Sistema usa la lista ordinata di oggetti (rapporto valore/peso decrescente);
- Inserisce un oggetto alla volta a ogni turno senza superare il peso massimo stabilito;
- Ripete fino a esaurimento oggetti validi.

Relazioni

- **include**: mostrano i passaggi obbligatori,
- **le note**: evidenziano i metodi core.

3.2 Diagramma delle Attività

I diagrammi di attività descrivono il flusso operativo dell'applicazione, illustrando i passaggi sequenziali e le condizioni decisionali che guidano le azioni dell'utente e dell'algoritmo durante la partita.

3.2.1 Attività Principali

Il diagramma delle attività riportato in Figura 3.3 descrive in dettaglio il flusso operativo dell'applicazione educativa sviluppata per l'esplorazione del problema dello zaino (Knapsack Problem 0/1). Tale diagramma modella il comportamento dinamico dell'applicazione, evidenziando le azioni sequenziali che avvengono durante l'interazione utente e l'esecuzione automatizzata degli algoritmi.

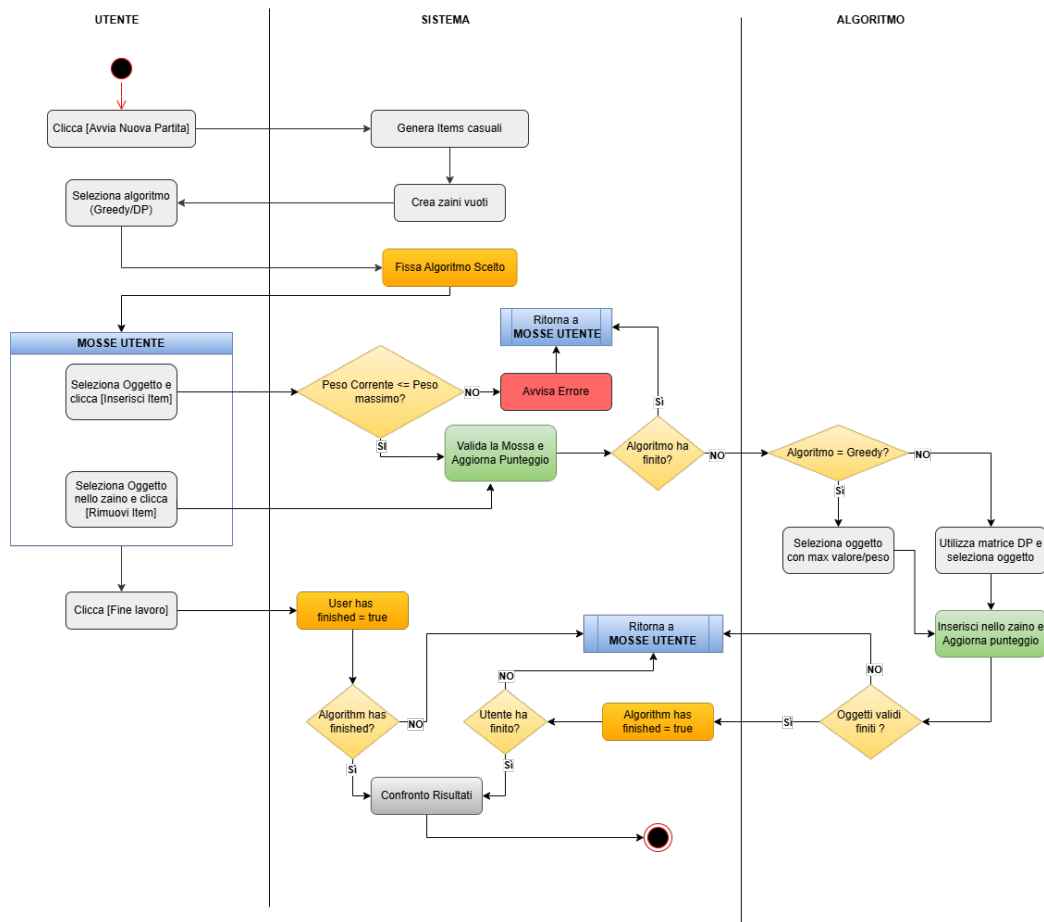


Figura 3.3: Diagramma delle Attività del Gioco sullo Zaino

Descrizione dettagliata del flusso

Il processo inizia con il clic sull'opzione *Avvia Nuova Partita*, che genera una lista di oggetti casuali (items) e inizializza due zaini vuoti, uno per l'utente e uno per la CPU. L'utente seleziona l'algoritmo desiderato tra **Greedy** e **Programmazione Dinamica (DP)**, che sarà poi utilizzato per l'esecuzione automatica della CPU.

Segue la fase di gioco vera e propria, in cui l'utente può:

- Selezionare un oggetto e inserirlo nello zaino cliccando su *Inserisci Item*.
- Selezionare un oggetto già presente nello zaino e cliccare su *Rimuovi Item*.
- Terminare la propria sessione con il pulsante *Fine Lavoro*.

Ad ogni azione dell'utente, il sistema verifica:

- Se il peso totale non eccede la capacità massima dello zaino.
- Se l'azione è valida, aggiornando in tal caso il punteggio complessivo.
- In caso contrario, avvisa l'utente con un messaggio d'errore.

Dopo il turno dell'utente, si avvia il turno della CPU che segue il comportamento deterministico dell'algoritmo selezionato:

- Se Greedy, viene selezionato l'oggetto con il rapporto valore/peso più alto tra quelli disponibili.
- Se DP, viene utilizzata una matrice per valutare le combinazioni ottimali e selezionare gli oggetti più vantaggiosi.

Ogni oggetto valido viene inserito nello zaino CPU e viene aggiornato il punteggio. Quando entrambi i giocatori (utente e algoritmo) hanno concluso le rispettive operazioni, si effettua il confronto finale dei risultati, determinando chi ha ottenuto il miglior valore complessivo.

Funzionalità evidenziate

Il diagramma mostra anche elementi fondamentali dell'interazione, come:

- Validazione dei vincoli (capacità massima dello zaino).
- Feedback immediato in caso di errore.
- Alternanza dei turni tra utente e CPU.
- Completamento del gioco con confronto dei punteggi.

Questa rappresentazione formale è utile per comprendere le dinamiche del sistema, supportare la fase di implementazione, validare i casi d'uso e facilitare la manutenzione futura del codice.

3.2.2 Diagramma delle Attività dell'Algoritmo CPU

Il diagramma in Figura 3.4 mostra il flusso dettagliato dell'attività dell'algoritmo selezionato dalla CPU. Dopo l'interazione dell'utente, l'algoritmo esegue la propria mossa automatica basandosi sull'approccio scelto: **Greedy** oppure **Programmazione Dinamica (DP)**.

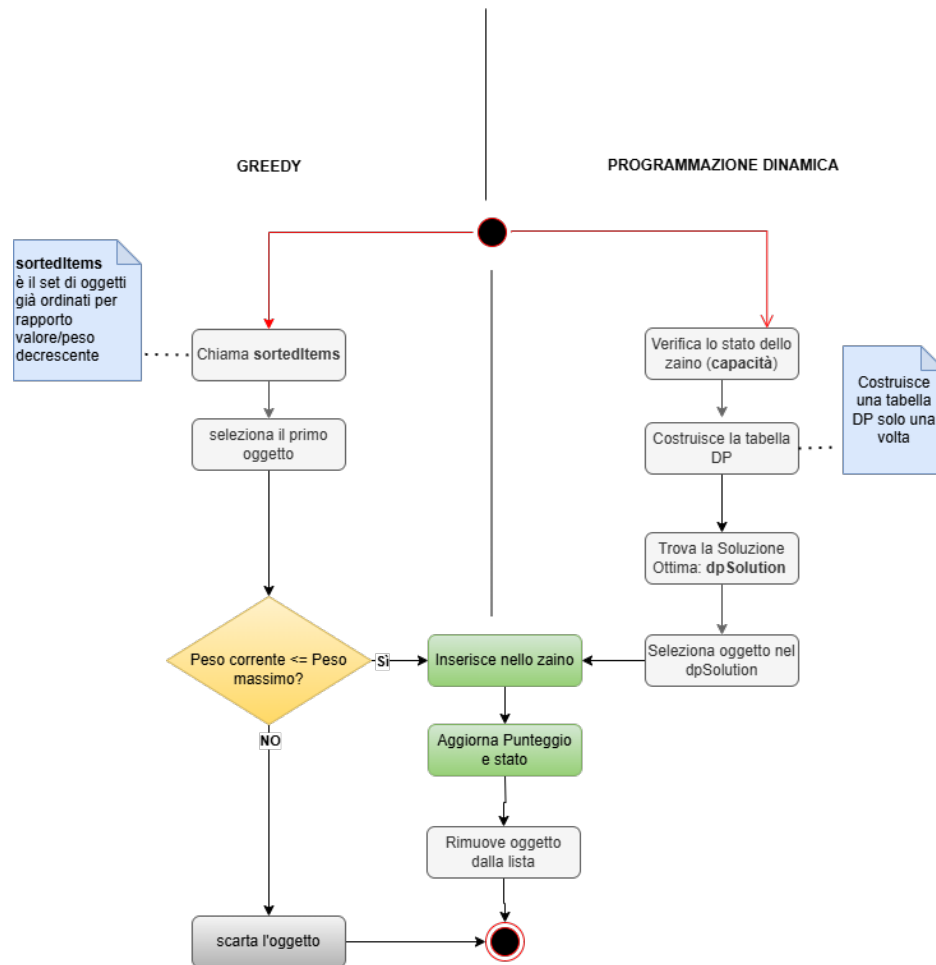


Figura 3.4: Diagramma delle Attività: Algoritmo CPU (Greedy / DP)

Caso Greedy

Nel ramo sinistro del diagramma si osserva il comportamento dell'algoritmo Greedy:

1. Viene chiamata la lista `sortedItems`, contenente gli oggetti ordinati per rapporto valore/peso decrescente.
2. L'algoritmo seleziona il primo oggetto disponibile.
3. Verifica se l'inserimento nello zaino rispetta il vincolo di capacità massima.
 - Se sì, inserisce l'oggetto nello zaino, aggiorna il punteggio e lo stato dello zaino.

- Se no, l'oggetto viene scartato.
4. L'oggetto viene rimosso dalla lista e si ripete il ciclo fino a quando non ci sono più oggetti validi.

Caso Programmazione Dinamica (DP)

Nel ramo destro del diagramma si evidenzia il comportamento dell'algoritmo DP:

1. Verifica la capacità residua dello zaino.
2. Costruisce una sola volta la matrice delle soluzioni (**DP table**).
3. Calcola la soluzione ottima a partire dalla tabella, ottenendo la sequenza di oggetti da inserire (**dpSolution**).
4. Seleziona l'oggetto corrente dalla soluzione ottima e procede all'inserimento nello zaino e all'aggiornamento dello stato, come nel caso Greedy.

Considerazioni finali

Questo diagramma mette in evidenza la differenza tra l'approccio **greedy (locale)** e quello **dinamico (globale)** nella risoluzione del problema dello zaino. Entrambi i percorsi sono strutturati per ripetere le azioni fino al completamento della soluzione, mantenendo la coerenza con lo stato corrente dello zaino e il rispetto dei vincoli imposti.

3.3 Diagrammi di Flusso delle Attività

In questa sezione sono presentati i diagrammi di flusso che illustrano le principali attività logiche e operative del gioco sviluppato. Ogni diagramma descrive un particolare aspetto del flusso dell'applicazione, dall'avvio della partita fino alla gestione delle mosse e degli algoritmi coinvolti.

3.3.1 Flusso Principale dell'Applicazione

Il diagramma in Figura 3.5 rappresenta il flusso principale dell'applicazione, dalla fase di inizializzazione fino alla conclusione della partita e alla visualizzazione dei risultati.

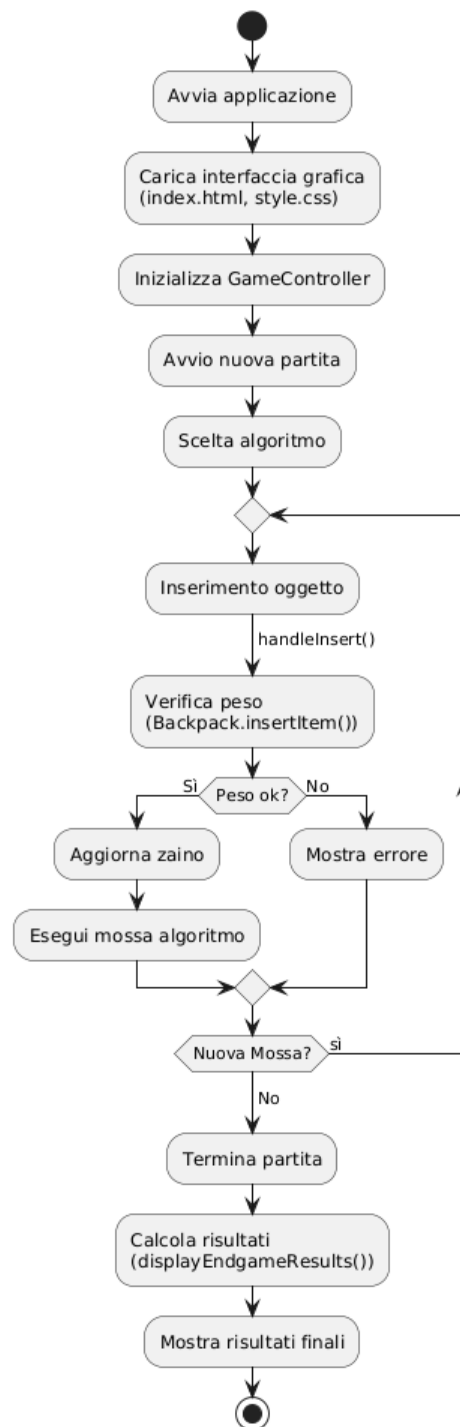


Figura 3.5: Flusso Principale dell'Applicazione

Descrizione del flusso

1. L'utente avvia l'applicazione, che carica l'interfaccia grafica (HTML e CSS).
2. Viene inizializzato il controller principale del gioco, `GameController`.
3. Dopo l'avvio di una nuova partita, viene scelta la strategia algoritmica.
4. L'utente effettua l'inserimento di un oggetto nello zaino. La funzione `handleInsert()` gestisce l'evento.
5. Lo zaino verifica se il peso complessivo è ancora accettabile (`Backpack.insertItem()`).
 - Se il peso è valido, lo zaino viene aggiornato e viene eseguita la mossa automatica dell'algoritmo.
 - Altrimenti, viene mostrato un messaggio di errore.
6. Si controlla se ci sono ulteriori mosse disponibili:
 - Se sì, il flusso torna al punto di inserimento.
 - Se no, la partita termina e viene calcolato il punteggio finale con `displayEndgameResults()`.

Questo diagramma sintetizza l'interazione continua tra utente e sistema, evidenziando il ciclo di gioco e le verifiche necessarie per il corretto proseguimento della partita.

3.3.2 Flusso dell'Algoritmo Greedy

La Figura 3.6 mostra il flusso delle operazioni eseguite dall'algoritmo Greedy, che seleziona gli oggetti in base al rapporto valore/peso in ordine decrescente.

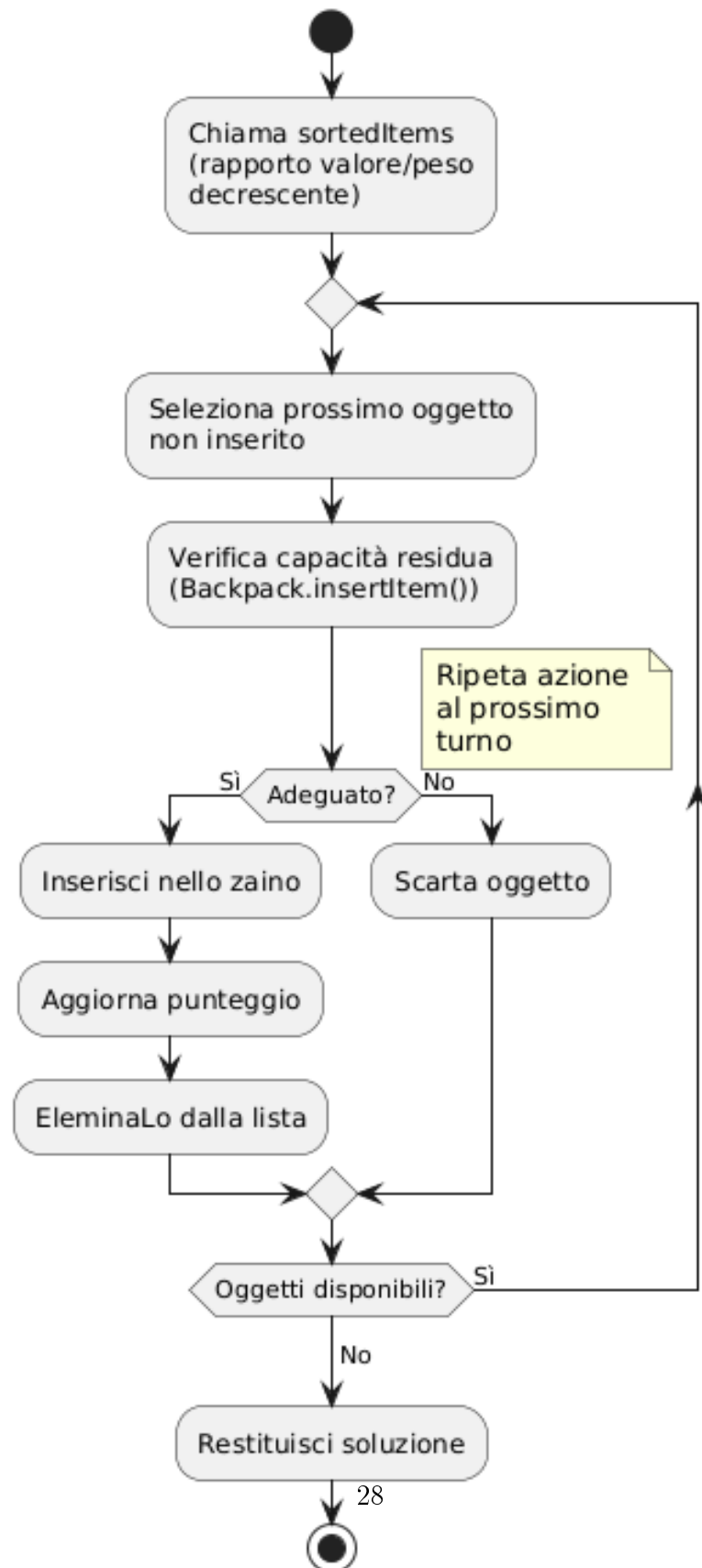


Figura 3.6: Flusso Algoritmo Greedy

Descrizione del flusso

Il diagramma mostra il funzionamento dell'algoritmo Greedy applicato al problema dello zaino. Il processo inizia con l'ordinamento degli oggetti disponibili in base al rapporto valore/peso in ordine decrescente. Questo criterio consente di dare priorità agli oggetti che offrono il miglior guadagno per unità di peso.

Successivamente, l'algoritmo esamina ogni oggetto nell'ordine stabilito, verificando se è possibile inserirlo nello zaino senza superare la capacità residua. Tale controllo viene eseguito tramite la funzione `Backpack.insertItem()` che valuta sia il peso complessivo sia la disponibilità di spazio nel contenitore.

Se l'oggetto può essere inserito, viene allocato nello zaino e il punteggio dell'algoritmo viene aggiornato. In caso contrario, l'oggetto viene scartato e si passa al successivo. Questo ciclo prosegue finché non vi sono più oggetti da esaminare. Al termine, viene restituita la configurazione finale ottenuta con l'approccio Greedy, che rappresenta una soluzione non necessariamente ottima, ma calcolata in modo molto efficiente.

3.3.3 Flusso dell'Algoritmo di Programmazione Dinamica

La Figura 3.7 rappresenta il flusso di esecuzione dell'algoritmo basato sulla Programmazione Dinamica (DP) per la risoluzione del problema dello zaino.

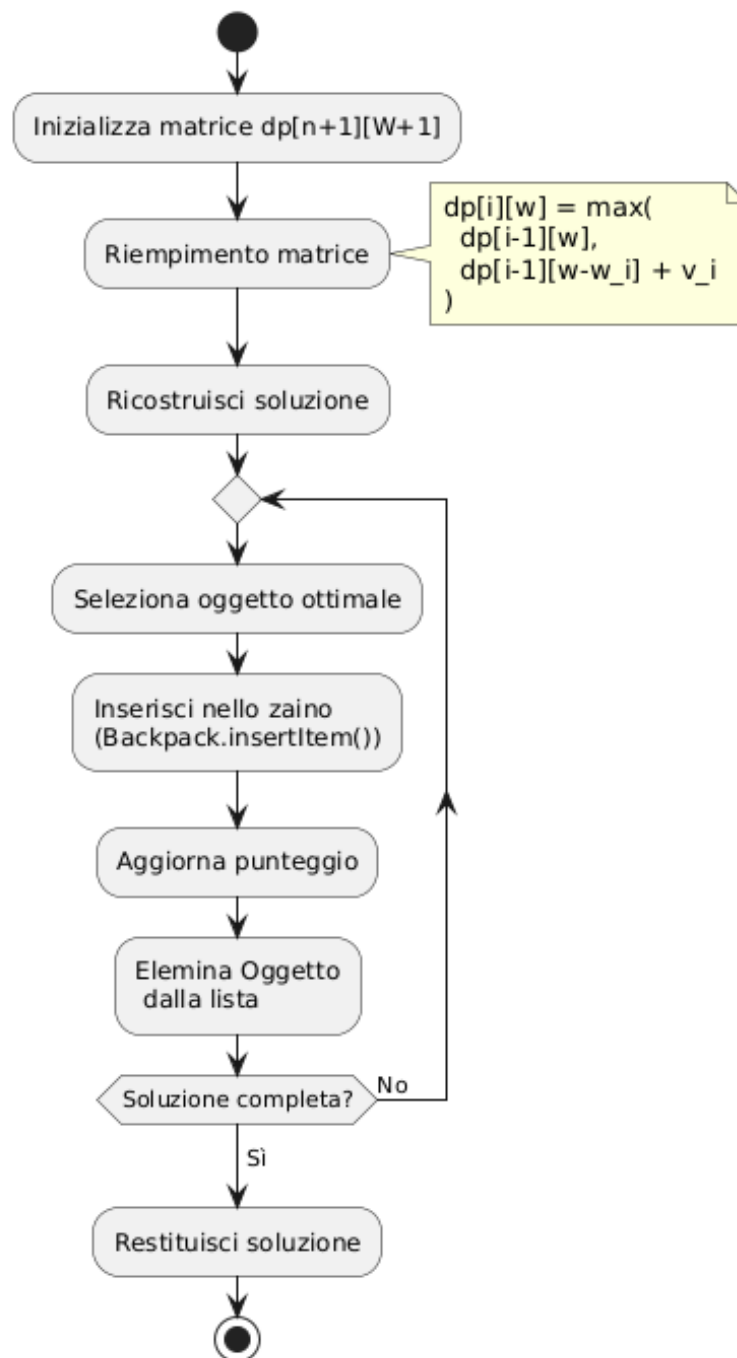


Figura 3.7: Flusso Programmazione Dinamica

Descrizione del flusso

Il diagramma rappresenta il flusso logico dell'algoritmo di Programmazione Dinamica (DP) applicato alla versione classica del problema dello zaino (0/1 Knapsack). In questo approccio, si costruisce una tabella bidimensionale $dp[n+1][W+1]$, dove n è il numero di oggetti e W la capacità massima dello zaino.

La matrice viene riempita iterativamente. Ogni cella $dp[i][w]$ rappresenta il massimo valore ottenibile considerando i primi i oggetti con capacità w . La formula usata riflette una scelta ottimale tra includere o meno l'oggetto corrente:

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i)$$

Dopo aver popolato la tabella, l'algoritmo ricostruisce la soluzione ottimale esaminando a ritroso le scelte effettuate. Per ogni oggetto incluso, viene eseguito l'inserimento nello zaino attraverso la funzione `Backpack.insertItem()`, aggiornando il punteggio e lo stato. Il ciclo si ripete finché tutti gli oggetti selezionati non sono stati allocati.

Infine, viene restituita la soluzione ottimale, garantita dal modello di programmazione dinamica, con il massimo valore raggiungibile dato il vincolo di capacità.

3.3.4 Flusso della Gestione dello Zaino

La Figura 3.8 mostra il flusso decisionale durante la fase di inserimento di un oggetto nello zaino. Viene verificata la possibilità di aggiungere l'oggetto sia in termini di peso totale che di spazio disponibile.

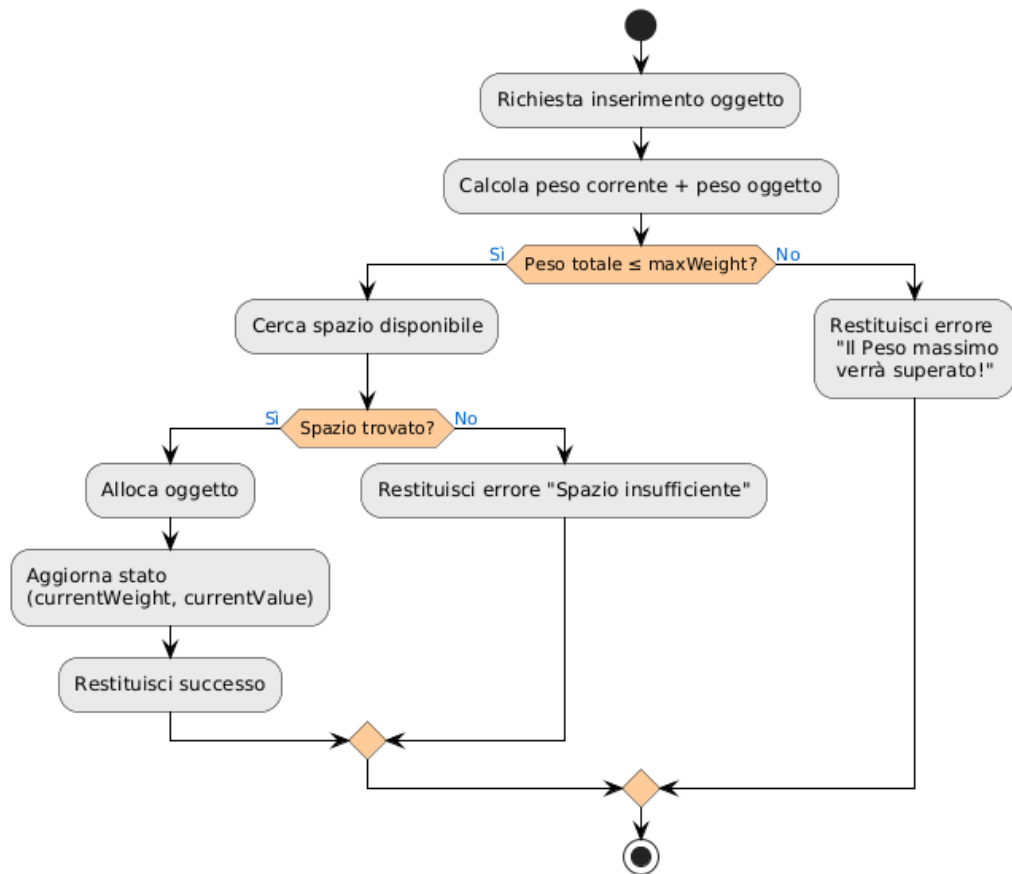


Figura 3.8: Flusso Gestione Zaino

Descrizione del flusso

Questo diagramma descrive il processo di gestione interna dello zaino nel momento in cui si tenta di inserire un oggetto, sia da parte dell'utente sia da parte dell'algoritmo.

Il flusso inizia con una richiesta di inserimento, che comporta il calcolo del nuovo peso totale come somma del peso corrente e di quello dell'oggetto selezionato. Se il peso totale rimane entro il limite massimo (es. 20 unità), si procede a cercare uno spazio disponibile nella griglia dello zaino, implementata come struttura a celle.

Se viene trovato uno spazio adeguato, l'oggetto viene fisicamente allocato nello zaino e vengono aggiornati i parametri `currentWeight` e `currentValue`, che rappresentano rispettivamente il peso totale e il valore accumulato fino a quel momento. Il metodo termina restituendo un successo.

In caso contrario, il sistema segnala un errore appropriato: “Spazio insufficiente” se il layout non consente il posizionamento, oppure “Peso eccessivo” se il peso supera la capacità. Questo meccanismo centralizzato garantisce il rispetto dei vincoli fisici e logici dello zaino.

3.4 Diagramma delle classi

Il diagramma delle classi presentato nella figura 3.9 illustra l’architettura logica del sistema sviluppato per simulare e visualizzare interattivamente la risoluzione del problema dello zaino, con il supporto di algoritmi come il Greedy e la Programmazione Dinamica.

Il sistema è modellato secondo un approccio orientato agli oggetti, con una chiara separazione tra i componenti responsabili del controllo del gioco, della logica degli utenti e degli algoritmi, della gestione dello zaino e della rappresentazione degli oggetti. Ogni classe ha precise responsabilità e collabora con le altre secondo relazioni di composizione e aggregazione.

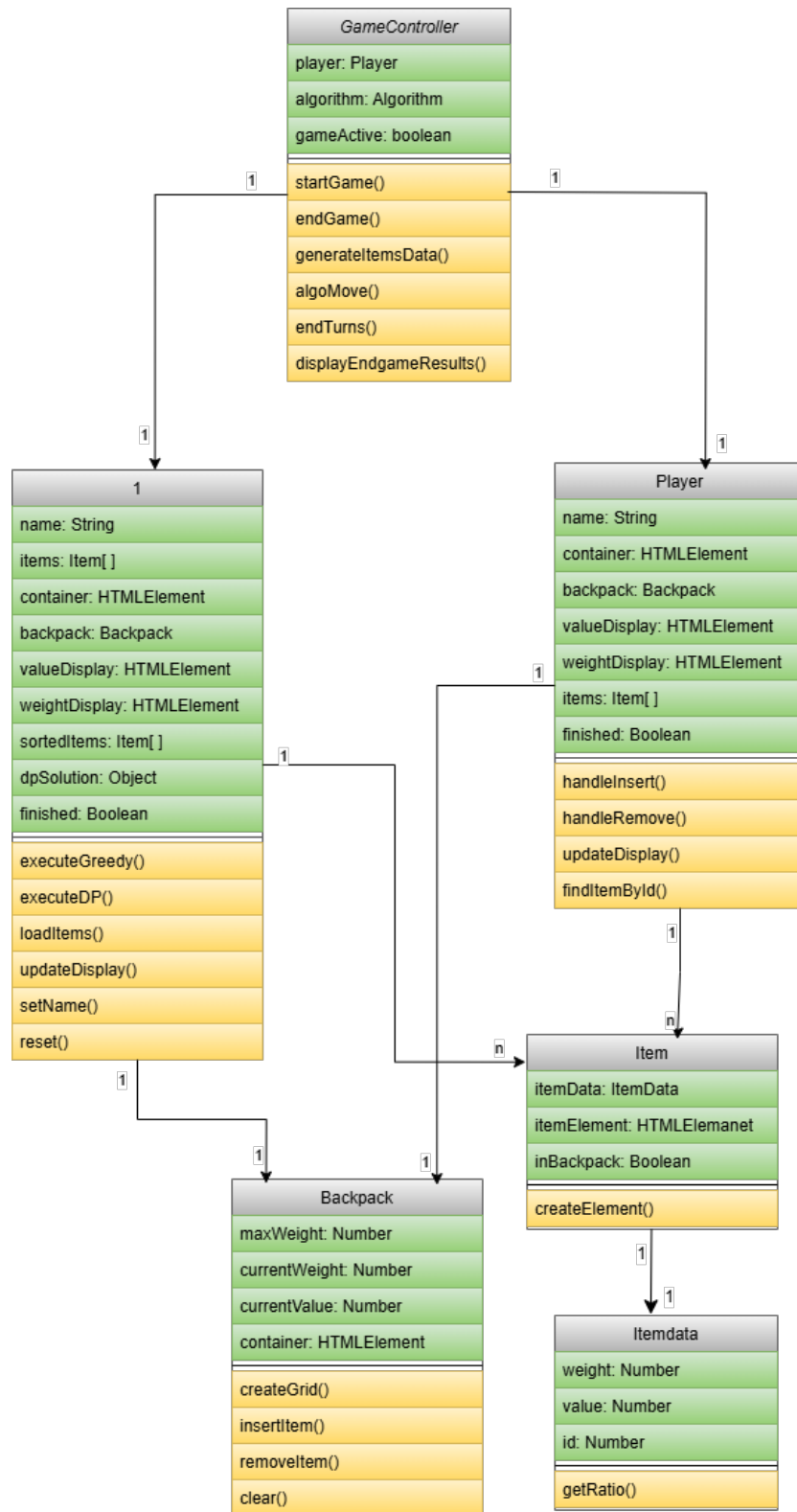


Figura 3.9: Classe Principale dell'Applicazione

- **GameController**: rappresenta il punto di ingresso e il coordinatore principale dell'applicazione. Questa classe controlla l'avvio (`startGame()`: Inizializza una nuova partita) e la chiusura del gioco (`endGame()`: Termina la partita e mostra i risultati), la generazione degli oggetti (`generateItemsDati()`), la gestione dei turni (`endTurns()`) e l'attivazione delle mosse dell'algoritmo (`algoMove()`). Al termine della partita, si occupa della visualizzazione dei risultati finali (`displayEndgameResults()`).
- **Player**: modella l'utente umano. Contiene informazioni relative agli oggetti posseduti (`items`), allo zaino personale, e gestisce le operazioni di inserimento e rimozione degli oggetti attraverso le funzioni `handleInsert()` e `handleRemove()`. Si occupa anche dell'aggiornamento della vista utente tramite `updateDisplay()`.
- **Algorithm**: rappresenta l'entità che esegue le strategie automatiche di risoluzione. Può eseguire sia l'algoritmo Greedy (`executeGreedy()`) sia quello basato su Programmazione Dinamica (`executeDP()`). Conserva una copia ordinata degli oggetti (`sortedItems`) e una rappresentazione interna della soluzione DP.
- **Backpack**: rappresenta lo zaino utilizzato sia dal giocatore sia dall'algoritmo. Tiene traccia del peso massimo (`maxWeight`), del peso corrente e del valore accumulato. Gestisce la griglia visiva (`grid`) e implementa le funzioni di inserimento e rimozione degli oggetti, oltre alla gestione grafica (`createGrid()` e `clear()`).
- **Item**: rappresenta una singola istanza grafica di un oggetto. Contiene un riferimento alla struttura dati logica (`ItemData`) e un riferimento all'elemento HTML associato. La proprietà `inBackpack` indica se l'oggetto è stato già inserito. La funzione `createElement()` permette di generare il componente visivo associato.
- **ItemData**: è la classe che contiene i dati astratti di un oggetto: peso, valore e identificativo. Fornisce una funzione `getRatio()` che calcola il rapporto valore/peso, utile per l'algoritmo Greedy.

Relazioni tra classi

Il diagramma mostra le seguenti principali relazioni strutturali tra le classi:

- **GameController** contiene un'istanza di **Player**, una di **Algorithm** e due istanze di **Backpack** (una per ciascun partecipante). Ciò sottolinea il suo ruolo centrale nella gestione del flusso del gioco.

- **Player** possiede una collezione di **Item** e utilizza uno **Backpack**, di cui può richiedere operazioni di modifica.
- **Algorithm** elabora una propria lista di **Item**, che può ordinare o filtrare, e anch'esso utilizza una propria istanza di **Backpack** per simulare le sue scelte.
- Ogni **Item** ha un'associazione 1:1 con una struttura dati **ItemData**, dalla quale eredita i valori fondamentali per le elaborazioni.

L'architettura così modellata permette una chiara separazione delle responsabilità, favorendo il riuso del codice e una gestione semplificata delle interazioni. L'utilizzo di questo pattern è evidente nella distinzione tra i dati (**ItemData**, **Backpack**), la logica (**GameController**, **Algorithm**) e la parte visuale (contenuta nei metodi delle classi).

Tale struttura ha favorito l'estendibilità del progetto, ad esempio con la possibilità di aggiungere facilmente altri algoritmi o modalità di gioco, e ne ha reso il codice mantenibile e testabile.

3.5 Diagrammi di Sequenza

I diagrammi di sequenza rappresentano le interazioni temporali tra le istanze delle classi durante l'esecuzione del sistema. Essi permettono di analizzare il comportamento dinamico del software, evidenziando la sequenza temporale dei messaggi scambiati tra gli oggetti partecipanti e chiarendo il flusso di controllo nei diversi casi d'uso.

Nel contesto del gioco didattico sul problema dello zaino, sono stati realizzati cinque diagrammi di sequenza che descrivono le seguenti situazioni:

- l'avvio e l'esecuzione completa di una sessione di gioco (flusso principale),
- l'inserimento manuale di un oggetto da parte dell'utente,
- l'esecuzione della strategia greedy da parte dell'algoritmo,
- l'esecuzione della programmazione dinamica,
- la gestione della fine partita e del punteggio finale.

Tali diagrammi aiutano a comprendere l'interazione tra il controller principale, il giocatore, l'algoritmo, lo zaino e gli oggetti durante il ciclo di vita dell'applicazione.

3.5.1 Sequenza Principale del Gioco

Questo diagramma di sequenza descrive il flusso completo dell'interazione principale tra i componenti nel corso di una partita tipica.

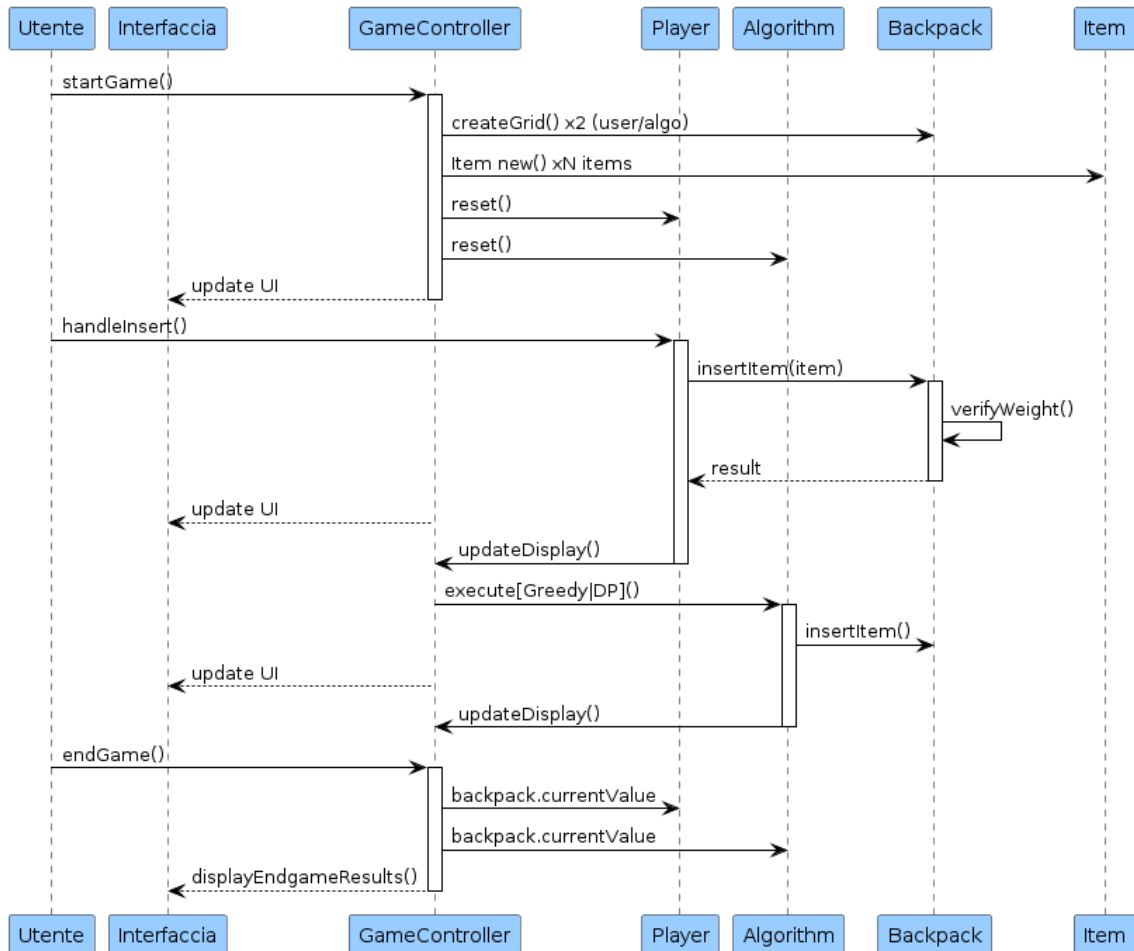


Figura 3.10: Sequenza Principale dell'Applicazione

Descrizione dettagliata:

- **Avvio della partita:** L'utente invoca il metodo `startGame()` sul `GameController`, che funge da coordinatore centrale. Questo inizializza le griglie dello zaino per entrambi i partecipanti (utente e algoritmo) tramite due chiamate a `createGrid()`.
- **Generazione degli oggetti:** Il `GameController` crea istanze della classe `Item`, ciascuna delle quali rappresenta un oggetto con peso e valore.

- **Reset degli stati:** Entrambi i partecipanti (`Player` e `Algorithm`) vengono resettati tramite chiamate ai rispettivi metodi `reset()`, per garantire una condizione iniziale pulita.
- **Renderizzazione UI:** Una volta completata l'inizializzazione, viene notificato l'utente affinché possa visualizzare l'interfaccia grafica aggiornata (`update UI`).
- **Turno utente:** L'utente inserisce un oggetto tramite `handleInsert()`. Il `Player` chiama `insertItem()` sul proprio `Backpack`, che verifica i vincoli di peso (`verifyWeight()`). Il risultato (successo o fallimento) viene ritornato al `Player`, che aggiorna l'interfaccia tramite `updateDisplay()`.
- **Turno algoritmo:** Il `GameController` invoca l'algoritmo selezionato (Greedy o DP) tramite `execute[Greedy|DP]()`. L'`Algorithm` esegue le sue mosse, inserendo oggetti nel proprio zaino e notificando i cambiamenti visivi con `updateDisplay()`.
- **Fine della partita:** L'utente invoca `endGame()`, che porta il `GameController` a interrogare entrambi i partecipanti per ottenere il valore totale del contenuto dello zaino. Infine, i risultati vengono mostrati all'utente tramite `displayEndgameResults()`.

Il diagramma evidenzia l'alternanza delle azioni tra utente e algoritmo, e mostra chiaramente il ruolo centrale del `GameController` nell'orchestrare l'intero processo di gioco, dalla fase di setup fino al calcolo e visualizzazione dei risultati.

3.5.2 Sequenza Inserimento Oggetto

Questo diagramma di sequenza approfondisce nel dettaglio il flusso di esecuzione che si verifica quando l'utente seleziona un oggetto e tenta di inserirlo manualmente nel proprio zaino. Viene così illustrata l'interazione tra le entità `Item`, `Player` e `Backpack` durante l'operazione di inserimento.

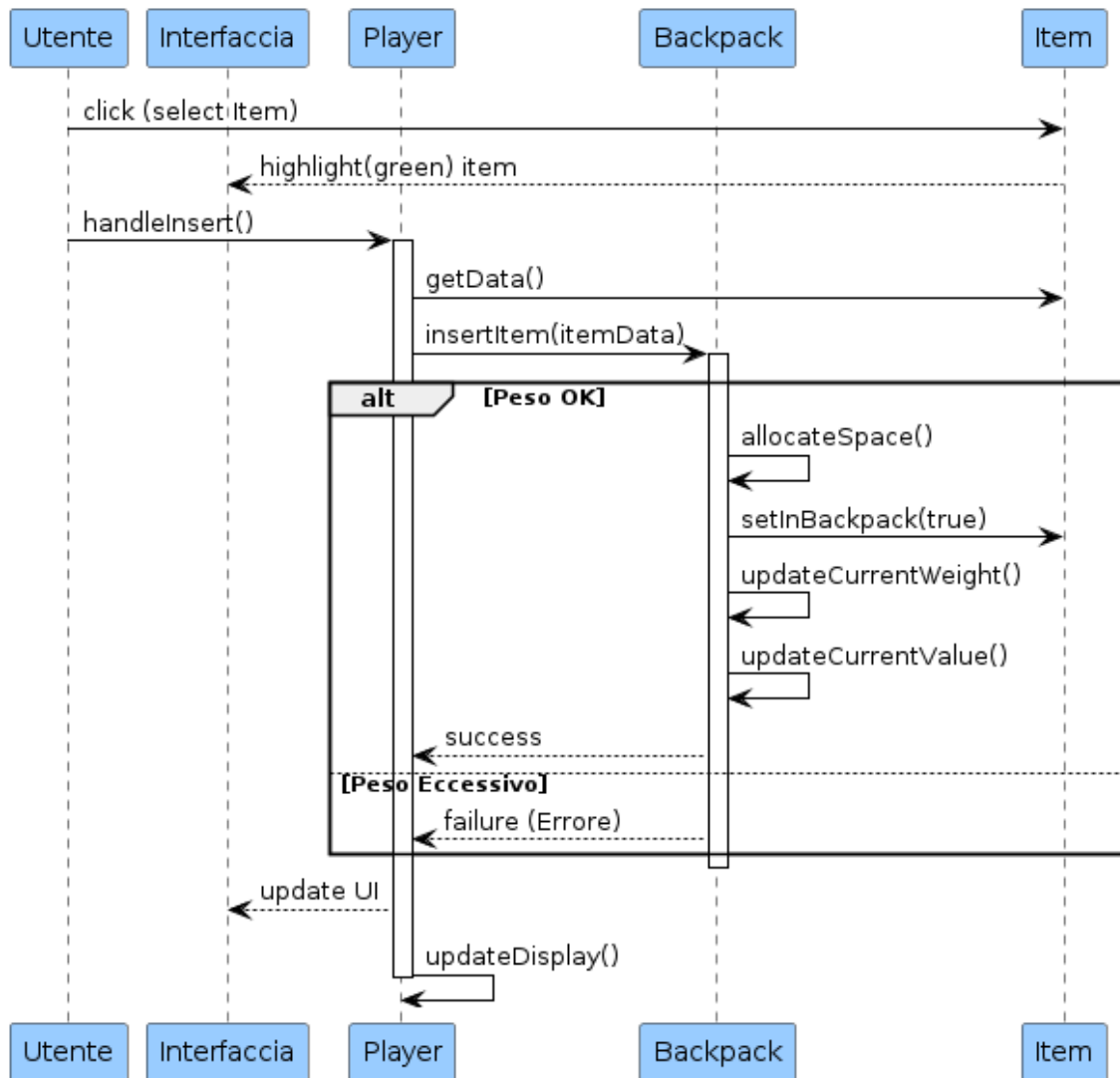


Figura 3.11: Sequenza di Inserimento Oggetto da Parte Utente

Descrizione dettagliata:

- **Selezione dell'oggetto:** L'utente compie un clic su un oggetto a schermo, generando un evento che attiva il metodo di selezione sull'istanza `Item`. L'oggetto risponde evidenziandosi visivamente (ad esempio cambiando bordo o colore), per indicare all'utente che è selezionato.
- **Richiesta di inserimento:** Dopo la selezione, l'utente invoca l'azione `handleInsert()` tramite il `Player`. Questo metodo recupera i

dati dell'oggetto tramite `getData()` e procede a chiamare il metodo `insertItem(itemData)` sullo zaino.

- **Verifica e allocazione:** Il **Backpack** verifica se l'aggiunta dell'oggetto rispetta i limiti di peso. Se il peso totale resta entro la soglia:
 - viene allocato uno spazio nella griglia,
 - l'oggetto viene marcato come “inserito nello zaino” con `setInBackpack(true)`,
 - si aggiornano peso e valore correnti dello zaino.
- In caso contrario, se l'aggiunta comporta il superamento del peso massimo, l'operazione viene rigettata con un messaggio di fallimento.
- **Aggiornamento dell'interfaccia:** In entrambi i casi, il **Player** aggiorna l'interfaccia utente tramite `updateDisplay()`, per riflettere la riuscita o meno dell'inserimento.

Questo diagramma è utile per comprendere come vengono applicati vincoli logici (peso massimo, disponibilità di spazio) in fase di interazione manuale da parte dello studente, simulando così una decisione computazionale con feedback immediato.

3.5.3 Sequenza dell'Algoritmo Greedy

Il diagramma mostra in dettaglio il funzionamento dell'algoritmo Greedy all'interno del flusso di gioco, evidenziando le interazioni tra le classi coinvolte nel processo automatico di ottimizzazione basato sul rapporto valore/peso.

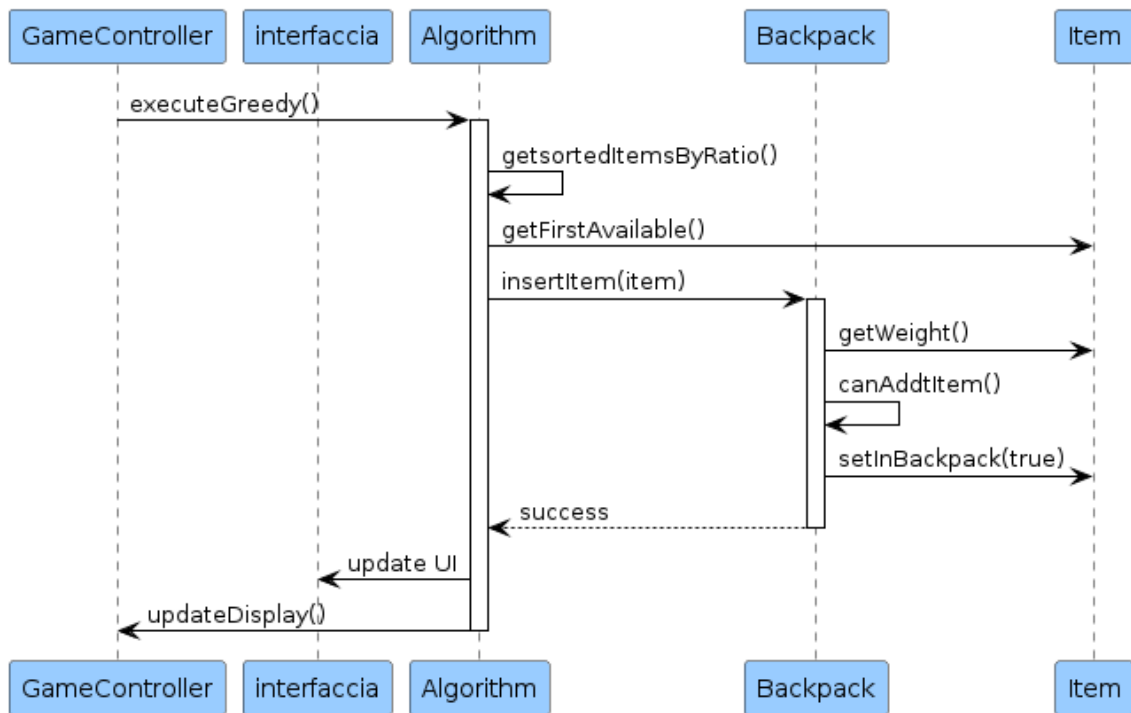


Figura 3.12: Sequenza di Esecuzione dell'Algoritmo di Greedy

Descrizione dettagliata:

- **Avvio dell'algoritmo:** Il `GameController` invoca il metodo `executeGreedy()` dell'entità `Algorithm`, attivando l'algoritmo greedy come turno automatico.
- **Ordinamento:** L'istanza `Algorithm` ordina gli oggetti a disposizione in ordine decrescente di rapporto valore/peso tramite il metodo `sortItemsByRatio()`.
- **Selezione e inserimento:** Dopo l'ordinamento, viene selezionato il primo oggetto disponibile (non ancora inserito) e si tenta l'inserimento nel `Backpack`.
- **Verifica dei vincoli:**
 - Lo zaino interroga l'oggetto per ottenerne il peso con `getWeight()`.
 - Verifica se il peso residuo disponibile consente l'inserimento con `canFitItem()`.

- Se compatibile, l'oggetto viene marcato come inserito (`setInBackpack(true)`), e viene restituito un esito positivo.
- **Aggiornamento interfaccia:** Infine, l'algoritmo comunica al `GameController` l'aggiornamento dello stato grafico tramite `updateDisplay()`, per mostrare il risultato dell'azione appena eseguita.

Questo diagramma rappresenta un esempio tipico di comportamento automatico orientato alla massimizzazione del valore all'interno di vincoli computazionali, come nel classico algoritmo greedy per il problema dello zaino.

3.5.4 Sequenza dell'Algoritmo di Programmazione Dinamica

Questo diagramma illustra le fasi operative dell'esecuzione dell'algoritmo di **Programmazione Dinamica (DP)** per la risoluzione del problema dello zaino, con evidenza delle chiamate tra componenti durante il turno dell'algoritmo.

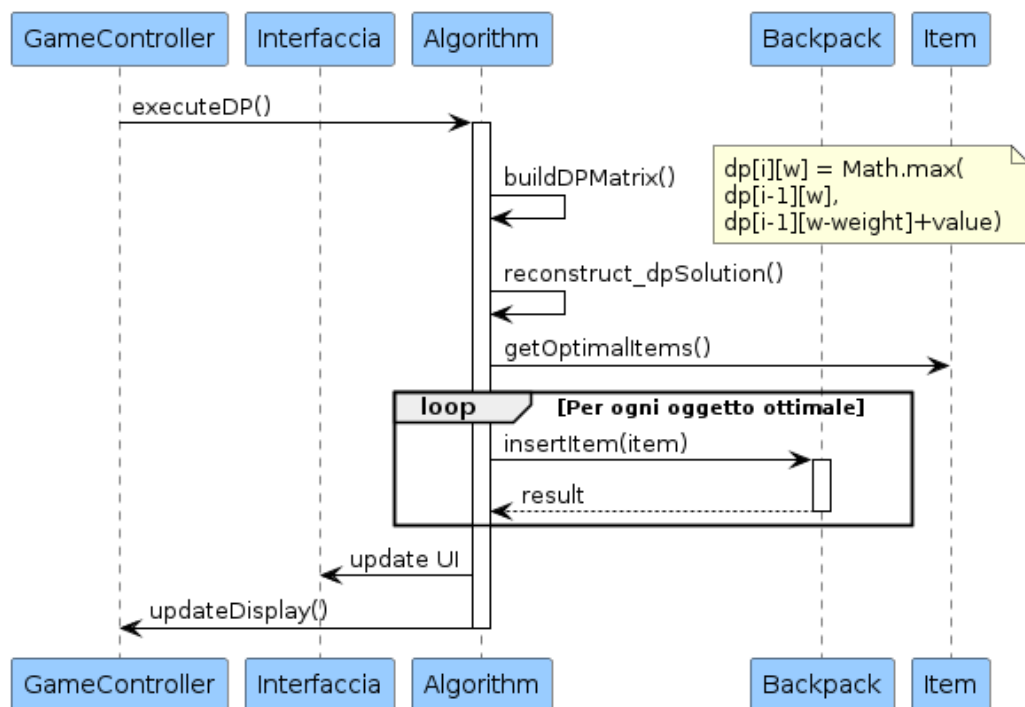


Figura 3.13: Sequenza di Esecuzione dell'Algoritmo di Programmazione Dinamica

Descrizione dettagliata:

- **Avvio dell'algoritmo:** Il `GameController` invoca il metodo `executeDP()` della classe `Algorithm`, che gestisce internamente l'intera procedura.
- **Costruzione della matrice:** L'algoritmo costruisce una matrice $dp[i][w]$ dove ogni cella rappresenta il valore massimo ottenibile con i primi i oggetti e una capacità residua w . La regola di transizione è:

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w - \text{weight}[i]] + \text{value}[i])$$

gestita dal metodo `buildDPMatrix()`.

- **Ricostruzione della soluzione ottimale:** Una volta completata la matrice, l'algoritmo utilizza `reconstructSolution()` per risalire agli oggetti che compongono la soluzione ottimale, recuperati attraverso `getOptimalItems()`.
- **Inserimento automatico nello zaino:** Per ogni oggetto selezionato, l'algoritmo tenta l'inserimento nel `Backpack` utilizzando `insertItem(item)`. Ogni operazione attiva temporaneamente il `backpack` per verificare compatibilità e aggiornare lo stato.
- **Aggiornamento finale dell'interfaccia:** Terminato il processo, `Algorithm` notifica al `GameController` che la fase di esecuzione è completata, tramite la chiamata `updateDisplay()`.

Questo flusso descrive in dettaglio un processo deterministico e ottimale, tipico della programmazione dinamica, evidenziando la differenza con l'approccio greedy più approssimativo.

3.5.5 Sequenza di Fine Partita

Questo diagramma rappresenta la fase conclusiva del gioco, attivata quando l'utente decide di terminare la partita. Mostra l'interazione tra le principali componenti del sistema per il calcolo del punteggio finale, il confronto tra giocatore e algoritmo, e la visualizzazione dei risultati all'utente.

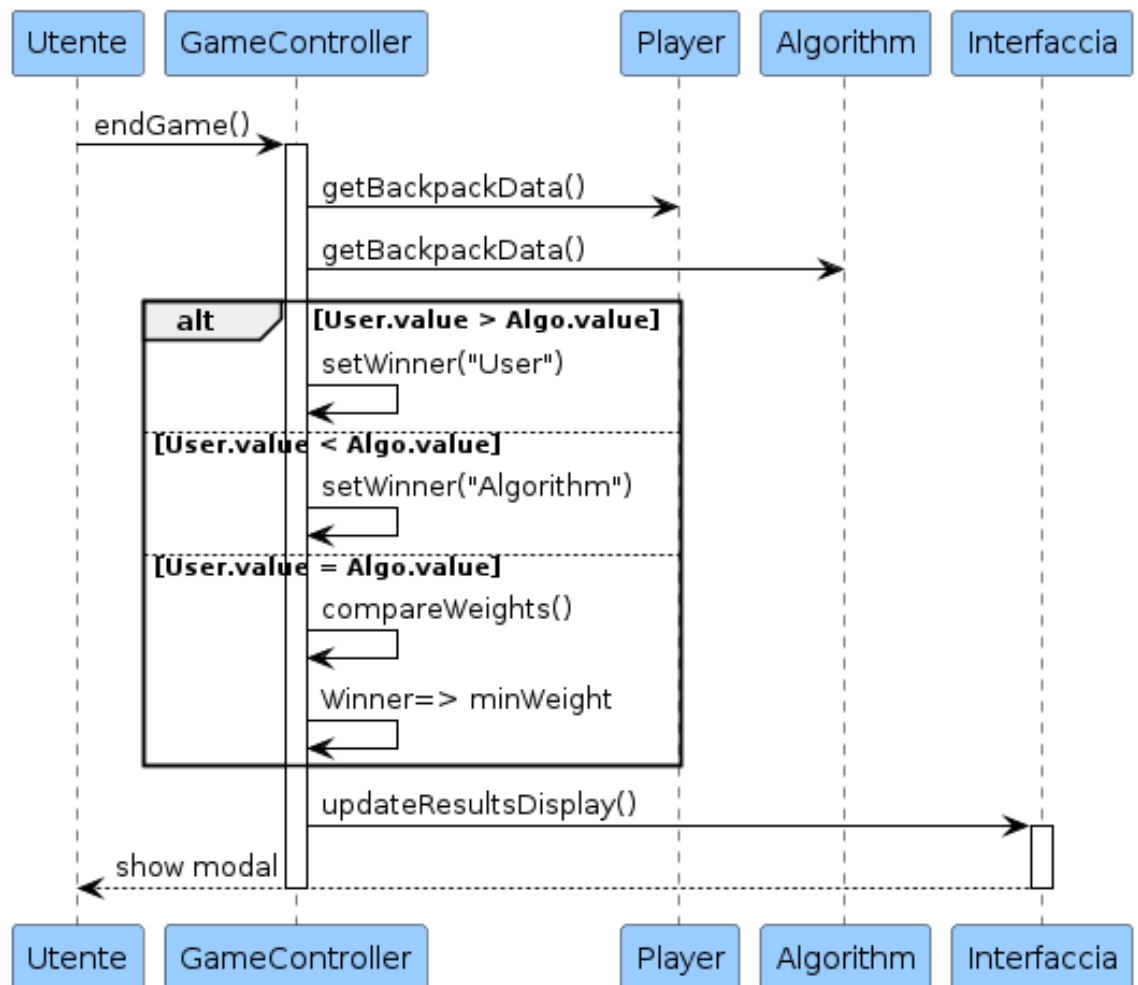


Figura 3.14: Sequenza di Fine Partia

Descrizione dettagliata:

- **Avvio del termine partita:** L'utente invoca il comando `endGame()` tramite l'interfaccia, che viene intercettato dal `GameController`.
- **Raccolta dati:** Il controller interroga sia il `Player` che l'`Algorithm` per recuperare i dati dei rispettivi zaini, come valore totale e peso.
- **Determinazione vincitore:**
 - Se il valore totale degli oggetti inseriti dall'utente è maggiore di quello dell'algoritmo, l'utente è dichiarato vincitore.
 - Se è minore, vince l'algoritmo.

- In caso di parità di valore, viene attivata una logica di confronto secondario basata sul peso totale (favorendo il minor peso).
- **Visualizzazione risultati:** il confronto viene gestito internamente tramite le chiamate `setWinner()` e `compareWeights()`. Dopo aver determinato l'esito, il `GameController` invia una richiesta al `DOM` per aggiornare l'interfaccia grafica, visualizzando una finestra modale con i risultati finali. L'utente riceve così il feedback visivo del risultato.

Questa sequenza rappresenta un'importante chiusura del ciclo interattivo, sintetizzando i dati raccolti durante la partita in un'esposizione chiara e immediata del vincitore, valorizzando l'aspetto ludico e competitivo dell'esperienza.

3.6 Diagrammi dei Componenti

In questa sezione vengono presentati i diagrammi dei componenti principali che costituiscono l'architettura dell'applicativo sviluppato. Tali diagrammi rappresentano visivamente l'organizzazione dei moduli software e le dipendenze tra essi, fornendo un quadro di insieme utile per comprendere la struttura e il flusso di interazione tra i diversi sottosistemi.

L'approccio adottato si basa su una **struttura multilivello**, che riflette le responsabilità logiche e funzionali dei componenti. Tale scelta è motivata dalla volontà di garantire un'elevata modularità, favorire la separazione delle responsabilità e facilitare le attività di manutenzione, test e futura estensione del progetto.

La Figura 3.15 mostra una rappresentazione architetturale a tre livelli del sistema: **Presentation Layer**, **Business Logic Layer** e **Data Layer**. Ogni livello contiene componenti specializzati, incaricati di una porzione ben definita della logica complessiva dell'applicazione.

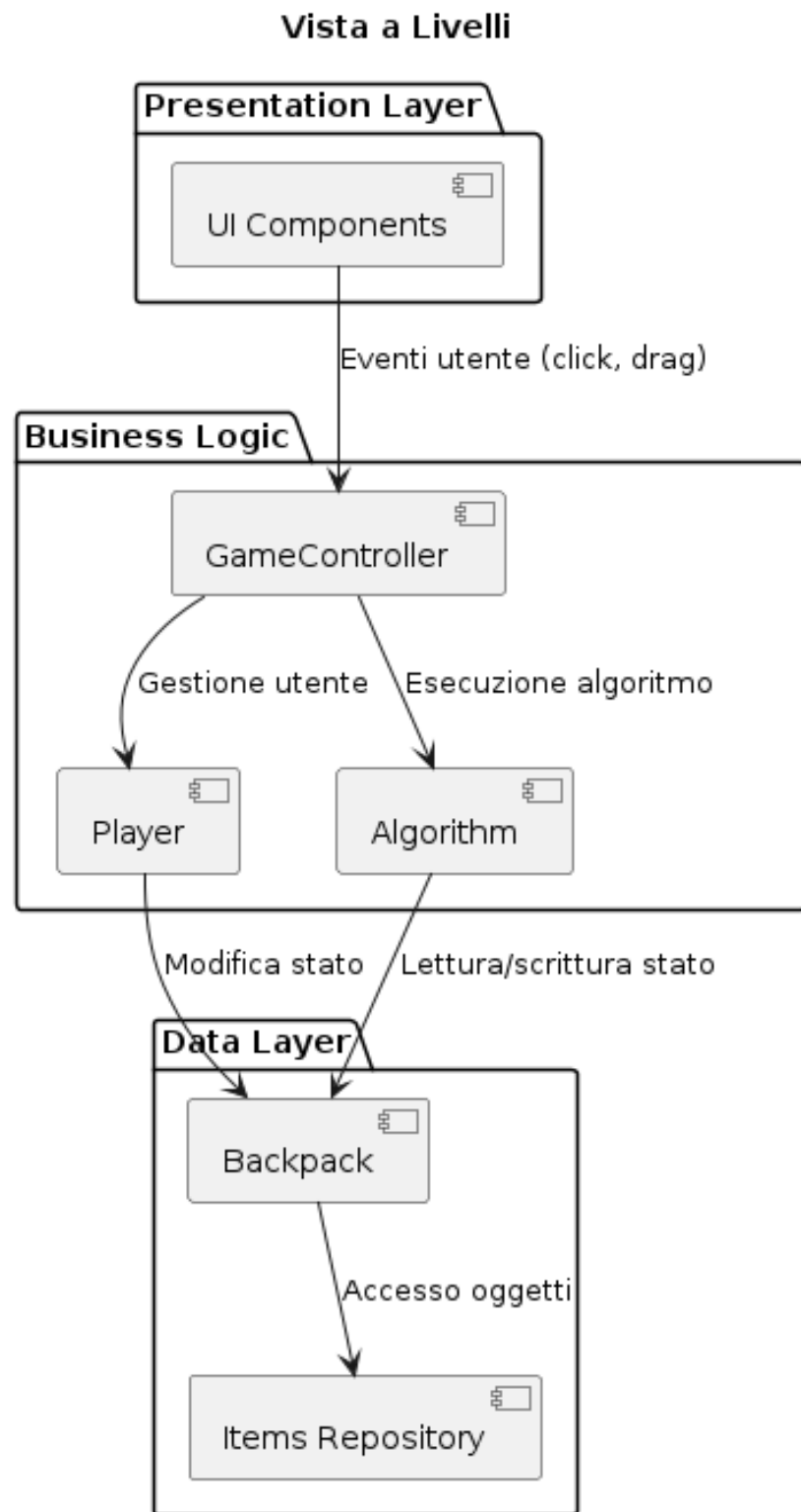


Figura 3.15: Vista a livelli dell'architettura dell'applicativo

3.6.1 Presentation Layer

Il **Presentation Layer** (o livello di presentazione) gestisce l'interazione tra l'utente e l'applicazione. Comprende:

- **UI Components:** è il componente che contiene gli elementi dell'interfaccia grafica (DOM), come pulsanti, contenitori degli oggetti, griglie per lo zaino, messaggi e popup. Si occupa di catturare gli eventi generati dall'utente (click, selezioni, ecc.) e di mostrare i risultati provenienti dalla logica sottostante.

3.6.2 Business Logic Layer

Il **Business Logic Layer** è il livello intermedio che implementa il comportamento dinamico dell'applicazione. Al suo interno troviamo:

- **Game Controller:** è il nucleo centrale che coordina le operazioni del sistema. Innesca le fasi del gioco, gestisce la transizione tra i turni e comunica sia con l'interfaccia utente sia con i moduli logici sottostanti.
- **Player Logic:** contiene la logica di gioco associata al giocatore umano. Controlla l'inserimento e la rimozione degli oggetti dallo zaino, nel rispetto delle regole del problema (es. peso massimo).
- **Algorithm Engine:** gestisce l'intelligenza artificiale del gioco. Implementa algoritmi per la risoluzione del problema dello zaino, come la strategia greedy e la programmazione dinamica. Opera in modo autonomo e interagisce con lo stato interno dello zaino.

3.6.3 Data Layer

Il **Data Layer** ha il compito di gestire lo stato interno dell'applicativo e l'accesso ai dati persistenti. Comprende:

- **Backpack State:** rappresenta lo zaino virtuale. Tiene traccia del peso totale, del valore complessivo, della configurazione della griglia e dello spazio residuo. È aggiornato dinamicamente in base alle azioni del giocatore e dell'algoritmo.
- **Items Repository:** è l'insieme degli oggetti disponibili per essere inseriti nello zaino. Ogni item ha proprietà quali peso, valore e identificativo unico. È da qui che il Game Controller attinge per la generazione degli oggetti iniziali.

La struttura a livelli proposta permette di mantenere separate le responsabilità tra gestione della logica, interfaccia e stato dell'applicazione. Ciò rende il codice più facilmente manutenibile, scalabile e testabile, ponendo solide basi architetturali per future estensioni o riutilizzi del progetto in ambiti didattici diversi.

3.7 Architettura del Progetto

Per consentire a una maggiore manutenibilità, riusabilità e testabilità dei componenti, facilitando l'estensione e la scalabilità del progetto nel tempo, è stato adottato un approccio ispirato al pattern **Model-View-Controller (MVC)**. In particolare l'architettura del sistema progettato segue un approccio stratificato a tre livelli, con l'obiettivo di separare chiaramente le responsabilità tra interfaccia utente, logica di business e gestione dei dati.

Inoltre, l'adozione di pattern architetturali consolidati, come il *Pattern Strategy* e la *Gestione dello Stato*, contribuisce a rendere il sistema modulare e facilmente adattabile a nuovi requisiti.

La figura 3.16 rappresenta graficamente la struttura generale del sistema, evidenziando l'interazione tra i componenti dei diversi livelli.

CAPITOLO 3. PROGETTAZIONE TRAMITE DIAGRAMMI UML

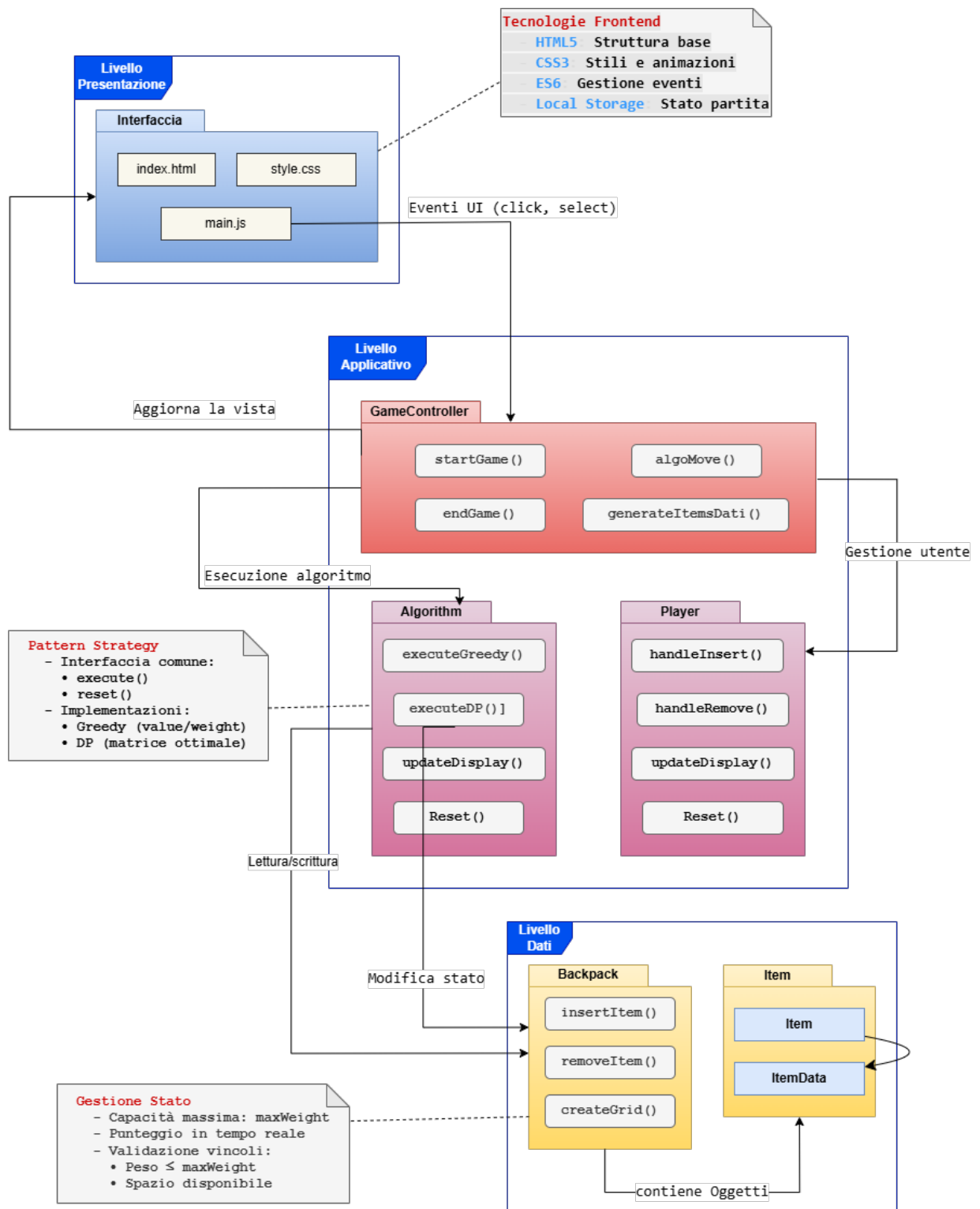


Figura 3.16: Architettura del Sistema

Il sistema è composto da tre livelli principali:

1. **Livello di Presentazione (Frontend)**: comprende i componenti responsabili della visualizzazione dell'interfaccia e della gestione degli eventi utente. I file coinvolti sono:

- `index.html` – struttura base della pagina.
- `style.css` – stile e layout.
- `main.js` – gestione dell'interazione e logica di inizializzazione.

I principali eventi gestiti sono click, selezioni e azioni dell'utente sulla griglia o sugli oggetti.

2. **Livello Applicativo (Business Logic)**: contiene la logica centrale del gioco. Il componente `GameController` coordina le attività tra le entità logiche:

- `GameController`: avvia e termina la partita, genera i dati degli oggetti e attiva le mosse algoritmiche.
- `Algorithm`: esegue le strategie di risoluzione (greedy, programmazione dinamica) attraverso l'interfaccia comune `execute()` del *Pattern Strategy*.
- `Player`: gestisce le azioni dell'utente, come inserimento e rimozione degli oggetti.

3. **Livello Dati (Modello)**: comprende le entità che rappresentano lo stato persistente del gioco.

- `Backpack`: modello dello zaino, con metodi per inserire/rimuovere oggetti e creare la griglia.
- `Item` e `ItemData`: rappresentano gli oggetti trasportabili, con peso, valore e dimensione.

Pattern utilizzati

Pattern Strategy Utilizzato nel modulo `Algorithm` per gestire le diverse strategie risolutive. L'interfaccia comune espone i metodi `execute()` e `reset()`, che vengono implementati dalle strategie Greedy e DP (Programmazione Dinamica).

Gestione dello Stato La classe `Backpack` e i metodi `startGame`, `endGame` mantengono lo stato coerente dello zaino e del gioco con:

- Capacità massima (`maxWeight`).
- Punteggio calcolato in tempo reale.
- Validazione dei vincoli di peso e spazio.
- `gameActive` per lo stato della partita.

Capitolo 4

Implementazione e Validazione

In questo capitolo viene descritta l'implementazione pratica dell'applicazione sviluppata per l'apprendimento del pensiero computazionale attraverso il problema dello zaino. Si fornisce una panoramica della struttura dei file che compongono il progetto e si illustrano i principali aspetti legati al codice HTML, JavaScript e CSS. Inoltre, si chiariscono le responsabilità dei vari componenti e come essi collaborano per fornire un'esperienza interattiva efficace.

4.1 Organizzazione del progetto

Il progetto è strutturato secondo una separazione chiara tra struttura (HTML), logica (JavaScript) e stile (CSS), seguendo i principi dell'architettura MVC (Model-View-Controller). I principali file che costituiscono il progetto sono:

- `index.html` – Definisce la struttura dell'interfaccia utente;
- `main.js` – Contiene la logica applicativa, le classi principali e la gestione degli eventi;
- `style.css` – Gestisce l'aspetto grafico dell'interfaccia tramite regole di stile CSS.

4.1.1 `index.html`

Il file `index.html` costituisce la base strutturale dell'applicazione. Utilizzando il linguaggio HTML5, definisce la griglia principale che contiene le diverse sezioni del gioco: l'area di gioco condivisa, i pannelli dell'utente e dell'algoritmo, e i controlli per la selezione e l'avvio della partita.

Gli elementi principali includono:

- Il contenitore con classe `container`, che definisce la disposizione a griglia delle sezioni tramite `CSS Grid`;
- La `game-space`, area in cui sono posizionati gli oggetti selezionabili (item) e lo zaino (backpack);
- I pannelli laterali `#algo-section` e `#user-section` per visualizzare rispettivamente le scelte dell'algoritmo e dell'utente;
- La sezione `.controls`, contenente i pulsanti e i menù a tendina per controllare il flusso del gioco.

L'HTML fornisce quindi una struttura semantica chiara, facilitando l'integrazione con lo stile e la logica, e migliorando l'accessibilità del contenuto.

4.1.2 `main.js`

Il file `main.js` rappresenta il cuore logico dell'applicazione. In esso sono implementate tutte le funzionalità dinamiche, la gestione degli eventi utente, l'inserimento e rimozione degli oggetti nello zaino, il calcolo del valore e del peso totale, e il confronto tra le prestazioni dell'utente e dell'algoritmo.

Le principali componenti di `main.js` includono:

- **Classi principali:** `ItemData`, `Item`, `Backpack`, `Player`, `Algorithm`, `GameController`, che seguono una logica ad oggetti per modellare entità e responsabilità;
- **Creazione degli oggetti:** gli oggetti di colori diversi vengono generati dinamicamente e rappresentati visivamente tramite celle (per il peso) e monete (coin per il valore);
- **Gestione delle interazioni:** selezione, inserimento e rimozione degli oggetti nello zaino sono gestiti tramite event listener, che aggiornano sia il modello interno sia il DOM;
- **Algoritmo automatico:** è presente una logica greedy per la scelta automatica degli oggetti, confrontabile con le scelte dell'utente;
- **Controllo del gioco:** l'intero flusso del gioco (inizializzazione, avvio partita, fine turno) è coordinato dalla classe `GameController`.

L'approccio orientato agli oggetti migliora l'organizzazione del codice, facilitando l'espansione futura verso algoritmi più complessi o nuove modalità di gioco.

4.1.3 style.css

Il file `style.css` si occupa dell'aspetto visivo dell'applicazione. Utilizzando un design responsive e pulito, migliora l'usabilità e l'esperienza utente.

Le principali caratteristiche grafiche definite nel CSS sono:

- Layout a griglia (**grid**) per organizzare in modo flessibile le aree di gioco e i pannelli;
- Colorazione differenziata per sezioni e oggetti: ad esempio, il colore giallo del backpack, il rosso per i messaggi di avviso, e l'oro per le monete (coin);
- Animazioni leggere per il feedback dell'interazione (hover, selezione);
- Classi dedicate alla visualizzazione coerente di elementi come le celle, i tooltip, le monete e i pulsanti.

Grazie a `style.css`, l'interfaccia risulta chiara, coerente e adatta a un contesto educativo, rendendo l'esperienza utente intuitiva e coinvolgente.

4.2 Interfaccia utente

L'interfaccia utente rappresenta il principale punto di contatto tra l'utente e il sistema, rivestendo un ruolo fondamentale nell'esperienza d'uso del gioco didattico sviluppato. È stata progettata per risultare intuitiva, reattiva e didatticamente efficace, con l'obiettivo di facilitare la comprensione del problema dello zaino (knapsack problem) e delle sue implicazioni algoritmiche.

La struttura visiva dell'applicazione è realizzata utilizzando HTML per la marcatura semantica, CSS per lo stile e la disposizione degli elementi, e JavaScript per la gestione dinamica dello stato del gioco e delle interazioni. L'interfaccia è pensata per guidare l'utente in ogni fase dell'esperienza interattiva, garantendo una chiara distinzione tra le aree funzionali e fornendo feedback continui.

4.2.1 Struttura generale

L'interfaccia è suddivisa logicamente in due sezioni principali:

- Una parte superiore, che introduce il gioco e ne mostra il titolo (“**SFIDA L’ALGORITMO**”);

- Una sezione centrale principale, racchiusa nel contenitore `<div class="container">`, che ospita due aree laterali speculari (**Sezione del giocatore**) dedicate al confronto tra algoritmo e utente; e un'area centrale (**Area di controllo**), che raccoglie i comandi e le informazioni sullo stato del gioco.

Le due aree laterali rappresentano rispettivamente a sinistra, l'area dell'**algoritmo**, ovvero la simulazione automatica che l'utente deve sfidare e a destra, l'area del **giocatore umano**, con cui l'utente interagisce direttamente. Ciascuna di queste sezioni è contenuta in un elemento HTML con classe `player-section` ed è suddivisa a sua volta in sotto-sezioni: una per la visualizzazione degli oggetti disponibili e una per la rappresentazione visiva dello zaino.

Sezione del giocatore: `class="player-section"`

Questa sezione consente all'utente di visualizzare prima

il **nome del giocatore**, mostrato nella parte superiore in blu della sezione; il contenitore degli oggetti selezionabili rappresentato dal blocco `<div class="itemContainer">`; poi lo zaino rappresentato da una griglia arancione nel blocco `<div class="backpack">`; e infine un pannello informativo (`<div class="info-panel">`) che riporta in tempo reale lo stato della soluzione proposta.

Gli oggetti disponibili sono rappresentati visivamente come **barre orizzontali colorate**, ciascuna delle quali ha:

- un **numero di celle proporzionale al peso** dell'oggetto;
- uno o più **cerchi gialli (coin)** al suo interno, proporzionali al valore (profitto) dell'oggetto;
- interattività: ciascun oggetto può essere selezionato cliccando sulla barra, che viene evidenziata per indicare lo stato attivo.

Ogni `itemContainer` può contenere nel nostro caso fino a sei oggetti contemporaneamente. Gli oggetti eventualmente esclusi dalla visualizzazione vengono caricati progressivamente man mano che altri vengono inseriti nello zaino.

Lo **zaino** è composto da una griglia di dimensione fissa, strutturata su due righe e dieci colonne (totale: 20 celle). Ciascuna cella, definita con la classe `.backpack-cell`, è:

- quadrata, con dimensione fissa;

- dotata di bordo grigio per facilitare la visualizzazione della struttura a griglia;
- posizionata in modo relativo per consentire l'inserimento dinamico degli oggetti.

Quando un oggetto viene inserito nello zaino, la sua barra viene ridimensionata per adattarsi alla griglia, occupando un numero di celle pari al suo peso. Gli oggetti vengono posizionati da sinistra verso destra e dall'alto verso il basso, seguendo una logica di riempimento spaziale.

Infine, il pannello `.info-panel` fornisce le seguenti informazioni:

- **Valore totale** degli oggetti attualmente presenti nello zaino;
- **Volume (peso) occupato** nello zaino, calcolato dinamicamente.

Entrambi i valori vengono aggiornati automaticamente dopo ogni interazione, tramite chiamate JavaScript, garantendo un feedback continuo e immediato.

Area di controllo: `class="controls"`

Questa sezione centrale raccoglie i comandi principali per l'interazione con il gioco:

- **Pulsante "Avvia Nuova Partita"**: reinizializza il gioco, generando un nuovo set casuale di oggetti identico sia dall'algoritmo che dall'utente e svuotando gli zaini.
- **Selettore algoritmo**: un menu a tendina che consente di scegliere l'algoritmo da sfidare (es. Greedy, Programmazione Dinamica).
- **Pulsante "Inserisci Item"**: consente di inserire lo specifico oggetto selezionato all'interno dello zaino.
- **Pulsante "Rimuovi Item"**: permette di rimuovere dallo zaino un oggetto selezionato.
- **Pulsante "Fine Lavoro"**: segnala che il giocatore ha terminato la propria strategia di inserimento.
- **Pulsante "Termina Partita"**: consente di interrompere anticipatamente la sessione in corso per ottenere i risultati della partita.

- **Area messaggi:** una zona testuale (`<div class="message">`) utilizzata per comunicazioni in tempo reale, come notifiche di errore (es. “Il peso massimo verrà superato!”) o messaggi di sistema (inizio/fine partita, ...).

4.2.2 Aspetti grafici e responsività

Lo stile dell’interfaccia è definito attraverso i fogli di stile `index.css` e `style.css`, ed è progettato secondo principi di chiarezza visiva, consistenza grafica e semplicità. Gli aspetti principali includono:

- **Palette di colori coerente:** utilizzo di verde per i pulsanti attivi e gli oggetti selezionati, il rosso per i messaggi di errore e arancione per indicare lo zaino.
- **Tipografia chiara:** font sans-serif, testi leggibili, dimensioni scalate secondo la gerarchia dell’interfaccia.
- **Organizzazione a griglia:** l’intero layout sfrutta il modello CSS Flexbox per posizionamenti fluidi e ordinati, sia in direzione orizzontale che verticale.
- **Transizioni ed effetti visivi:** evidenziazione degli elementi interattivi al passaggio del mouse o alla selezione, transizioni fluide per migliorare la percezione dell’interazione.
- **Responsività:** l’interfaccia è adattabile a schermi di dimensioni diverse (desktop e tablet), mantenendo leggibilità e proporzioni.

Nel complesso, l’interfaccia utente è stata realizzata seguendo i principi di usabilità, accessibilità e design didattico. Le rappresentazioni visive (barre, griglia, coin) permettono agli utenti di apprendere concetti complessi legati all’ottimizzazione spaziale e alla selezione di oggetti in modo intuitivo e interattivo. Questo rende l’applicazione adatta anche a studenti alle prime esperienze con l’informatica e la programmazione, promuovendo un apprendimento attivo basato sull’esplorazione e la sperimentazione.

4.3 Interazioni e funzionalità principali dell’applicazione

In questa sezione si descrivono le funzionalità interattive principali dell’applicazione, dal caricamento iniziale fino all’interazione con gli oggetti e alla

loro gestione nello zaino virtuale. Il codice sorgente fa riferimento alle classi implementate nel file `main.js`, strutturato secondo il paradigma MVC.

4.3.1 Inizializzazione e avvio del gioco

L'inizializzazione avviene tramite la creazione dell'istanza principale della classe `GameController`, che si occupa di orchestrare le varie componenti dell'applicazione: vista, logica del gioco, gestione degli oggetti e sincronizzazione con l'interfaccia utente.

```
1 const controller = new GameController();
2 startButton.addEventListener("click", () => controller.
  startGame());
```

Codice 4.1: Creazione del controller e avvio del gioco

Il metodo `startGame()` inizializza gli oggetti, la griglia dello zaino, gli elementi dell'interfaccia e imposta lo stato iniziale del gioco.

- Genera un insieme casuale di oggetti tramite la classe `ItemData`, utilizzando il metodo `generateItemsDati()` della classe `GameController`.
- Costruisce la vista iniziale dell'inventario e dello zaino attraverso rispettivamente il metodo `createElement()` della classe `Item`, e il metodo `reset()` delle classi `Player` e `Algorithm`.
- Abilita i pulsanti per l'interazione utente con il comando `toggleButtons(true)`.

4.3.2 Selezione dell'algoritmo da sfidare

L'utente può scegliere l'algoritmo da sfidare tramite l'interfaccia grafica. Il controller associa a ogni pulsante una funzione di callback che crea un'istanza della classe `Algorithm` con la strategia selezionata e della classe `Player`.

```
1 algorithmSelect = document.getElementById("
  algorithmSelect");
2 algorithmSelect.addEventListener("change", () => {
3   const selectedValue = algorithmSelect.value;
4   controller.setAlgorithm(selectedValue);
5   algorithmSelect.disabled = true;
6 });
```

Codice 4.2: Selezione dell'algoritmo

Dopo la selezione dell'algoritmo o di un item nel contenitore, il pulsante viene automaticamente disattivato e il metodo `setAlgorithm()` aggiorna lo stato interno del gioco e memorizza il tipo di algoritmo scelto, che verrà eseguito al momento opportuno.

4.3.3 Selezione e interazione con gli oggetti

Ogni oggetto è rappresentato da un'istanza della classe `Item` e visualizzato come una barra colorata all'interno del contenitore dell'inventario.

L'interazione si articola in due fasi:

1. **Selezione:** l'utente clicca sull'oggetto per evidenziarlo.
2. **Azione:** una volta selezionato, l'utente può inserirlo nello zaino o rimuoverlo, tramite i pulsanti dedicati.

La selezione è gestita dalla funzione `selectItem(element)`, che evidenzia visivamente l'elemento e lo memorizza nella variabile globale `selectedItem`.

```
1 function selectItem(element) {  
2     if (!element) return;  
3     if (!element.dataset.originalColor) {  
4         element.dataset.originalColor = element.style.  
5         backgroundColor;  
6     }  
7     element.style.backgroundColor = "lightgreen";  
8     element.classList.add("selected");  
9 }
```

Codice 4.3: Gestione della selezione di un oggetto

Una volta selezionato, l'oggetto può essere gestito tramite le funzioni `handleInsert()` o `handleRemove()`, attivate dalla pressione dei pulsanti `Inserisci Item` o `Rimuovi Item`.

4.3.4 Gestione dell'inserimento e rimozione nello zaino

Il metodo `handleInsert()` della classe `Player` controlla la disponibilità di spazio nello zaino e la validità dell'oggetto selezionato. Se le condizioni sono soddisfatte, il metodo delega a `insertItem()` della classe `Backpack` l'inserimento effettivo poi aggiorna la vista.

```
1 handleInsert() {  
2
```

```
3   const weight = parseInt(selectedItem.dataset.weight,
4   10);
5   const itemId = selectedItem.dataset.itemId;
6   const newWeight = this.backpack.currentWeight +
7   weight;
8
9   if (newWeight > this.backpack.maxWeight) {
10      this.messaggio1.textContent = "Attenzione! Il
11      peso massimo verrà superato";
12      return false;
13   }
14
15   const item = this.findItemById(itemId);
16   if (!item) {
17      console.warn("Item non trovato");
18      return false;
19   }
20
21   const inserted = this.backpack.insertItem(item, this.
22   container);
23   if (inserted) {
24      this.updateDisplay();
25      deselectItem(selectedItem);
26      selectedItem = null;
27      return true;
28   }
29   return false;
30 }
```

Codice 4.4: Inserimento nello zaino

Analogamente, la funzione `handleRemove()` permette di rimuovere un oggetto precedentemente inserito, aggiornando sia la vista che lo stato interno del gioco.

```
1 handleRemove() {
2
3   const itemId = selectedItem.dataset.itemId;
4   if (!itemId) return;
5
6   const item = this.findItemById(itemId);
7   if (!item || !item.inBackpack) {
8      console.warn('Elemento non trovato o non nello
9      zaino per itemId: ${itemId}');
10      return;
11   }
12 }
```

```
10     }
11
12     this.backpack.removeItem(item, this.container);
13     this.updateDisplay();
14     selectedItem = null;
15 }
```

Codice 4.5: Rimozione dallo zaino

Il layout dello zaino viene aggiornato dinamicamente per riflettere lo stato attuale. Le barre che rappresentano gli oggetti si dispongono da sinistra verso destra, con un riorganizzazione automatico in caso di spazio liberato.

4.3.5 Gestione dei turni e sincronizzazione

L'applicazione implementa una modalità a turni alternati: prima agisce l'utente, successivamente l'algoritmo. Il controllo della sequenza è gestito principalmente dai metodi `algoMove()` ed `endTurns()` della classe `GameController`, che vengono invocati dopo le interazioni dell'utente. Questi metodi mantengono lo stato corrente della partita e determinano quale componente (utente o algoritmo) è autorizzato ad agire in un determinato momento.

All'interno delle funzioni associate ai pulsanti di inserimento e rimozione:

- `insertBtn.addEventListener("click", () => {...});`
- `removeBtn.addEventListener("click", () => {...});`

si trovano i controlli fondamentali per la gestione dei turni. In particolare, al termine dell'azione dell'utente viene verificato se entrambi i partecipanti non hanno ancora terminato, viene programmata l'esecuzione dell'algoritmo dopo un breve ritardo temporale.

```
1  const successTurn = controller.player.handleInsert();
2  if (!successTurn) {
3      return ;
4  }
5  // nel caso di handleRemove() : controller.player.
   handleRemove();
6  if (!controller.player.finished && !controller.algorithm.
   finished) {
7      controller.messaggio1.textContent = "Algoritmo in
   esecuzione";
8      toggleButtons(false);
9      setTimeout(() => controller.algoMove(), 2000);
```

10 }

Codice 4.6: Gestione dei turni tra utente e algoritmo

Il ritardo di qualche secondo (ottenuto tramite `setTimeout`) è introdotto per simulare in modo fluido la successione dei turni e dei processi.

Quando l'utente preme il pulsante **Fine Lavoro**, segnalando di aver concluso le proprie scelte, viene invocato il metodo `endTurns()`, che gestisce automaticamente le mosse residue dell'algoritmo fino al termine del suo turno.

La sincronizzazione tra il modello dati e la rappresentazione grafica (vista) è assicurata dai metodi `updateDisplay()` e `appendChild()`, presenti rispettivamente nelle classi `Player` e `Backpack`. Tali metodi aggiornano dinamicamente l'inventario, lo zaino e le informazioni visive dell'interfaccia.

4.3.6 Esecuzione dell'algoritmo selezionato

Alla fine del turno dell'utente, l'algoritmo selezionato entra in azione. L'esecuzione è gestita dal metodo `algoMove()` della classe `GameController`, che richiama la strategia configurata nella classe `Algorithm`. Attualmente sono supportate strategie come Greedy e Programmazione Dinamica (DP), ma l'architettura è estendibile a ulteriori approcci euristici.

Durante l'esecuzione dell'algoritmo, i pulsanti `insertBtn` e `removeBtn` vengono disattivati, impedendo all'utente ulteriori interazioni.

```

1 algoMove() {
2   if (!this.gameActive || this.player.finished) return;
3
4   let success = false;
5
6   if (this.algorithm.name === "greedy") {
7     success = this.algorithm.executeGreedy();
8   } else if (this.algorithm.name === "dp") {
9     success = this.algorithm.executeDP();
10  }
11
12  if (!success) {
13    this.messaggio1.textContent = "L'algoritmo ha
finito!";
14    this.algorithm.finished = true;
15  }
16
17  toggleButtons(true);

```

18 }

Codice 4.7: Esecuzione del turno dell'algoritmo

Le funzioni `executeGreedy()` e `executeDP()`, appartenenti alla classe `Algorithm`, ricevono gli oggetti disponibili e lo stato attuale dello zaino. Esse calcolano la selezione ottimale degli item da inserire, aggiornando coerentemente sia il modello dati che la vista.

4.3.7 Conclusione della partita e valutazione finale

Quando entrambi i partecipanti (utente e algoritmo) hanno terminato le proprie mosse, il gioco giunge al termine. La valutazione finale è gestita dal metodo `displayEndgameResults()` del `GameController`, che calcola il punteggio complessivo in base al valore totale e al peso degli oggetti inseriti nello zaino.

```
1 displayEndgameResults() {
2     let message = "";
3     this.messaggio1.textContent = "La Partita è finita!";
4
5     const userValue = this.player.backpack.currentValue;
6     const algoValue = this.algorithm.backpack.
currentValue;
7     const userWeight = this.player.backpack.currentWeight
;
8     const algoWeight = this.algorithm.backpack.
currentWeight;
9
10    if (userValue > algoValue) {
11        message = "Complimenti! Hai vinto la partita con
un valore maggiore!";
12    } else if (userValue < algoValue) {
13        message = "L'algoritmo ha vinto! Ritenta per
migliorare la tua strategia.";
14    } else {
15        if (userWeight < algoWeight) {
16            message = "Stesso valore, ma hai usato meno
peso. Hai vinto!";
17        } else if (userWeight > algoWeight) {
18            message = "Stesso valore, ma l'algoritmo ha
usato meno peso. Ha vinto!";
19        } else {
20            message = "Pareggio perfetto!";
```

```
21     }
22 }
23
24 this.resultsContainer.innerHTML = '<h3>${message}</h3>';
25 this.resultsContainer.style.display = "block";
26 }
```

Codice 4.8: Valutazione finale e termine della partita

Il risultato viene visualizzato all'utente tramite un messaggio chiaro e motivante, inserito dinamicamente nel DOM.

4.3.8 Comunicazione e feedback all'utente

Durante tutto il flusso di gioco, l'applicazione fornisce feedback continui all'utente mediante messaggi testuali dinamici, evidenziazioni visive e la gestione dello stato interattivo dei pulsanti.

I messaggi principali sono visualizzati tramite aggiornamenti alle variabili DOM come `messaggio1` e `resultsContainer`:

```
1 this.messaggio1.textContent = "La Partita è iniziata!";
2 this.messaggio1.textContent = "attenzione! Il Peso
  massimo verrà superato";
3 this.messaggio1.textContent = "algoritmo in esecuzione";
```

Codice 4.9: Feedback informativo all'utente

L'interfaccia grafica utilizza anche meccanismi visivi, come la colorazione degli oggetti selezionati, il movimento animato degli elementi nello zaino e la scomparsa/ricomparsa dinamica dei componenti, per rendere l'esperienza più immediata e coinvolgente.

Infine, il controller regola l'interattività dei pulsanti attraverso il metodo `toggleButtons(enabled)`, che abilita o disabilita le azioni dell'utente in base al turno o allo stato del gioco.

```
1 toggleButtons(enabled) {
2     insertButton.disabled = !enabled;
3     removeButton.disabled = !enabled;
4 }
```

Codice 4.10: Controllo dei pulsanti dell'interfaccia

Questo approccio garantisce un flusso d'interazione coerente, riducendo errori e rendendo chiara all'utente la fase corrente del gioco.

4.4 Algoritmi Implementati

All'interno dell'applicazione sono stati implementati due algoritmi per la risoluzione del problema dello zaino: un approccio *greedy* e uno basato sulla *programmazione dinamica*. La loro integrazione nel gioco consente di confrontare in tempo reale le strategie adottate dall'utente con quelle automatiche, mettendo in evidenza vantaggi e limiti di ciascun approccio.

4.4.1 Algoritmo Greedy

L'algoritmo greedy adotta una strategia euristica basata sull'ordinamento degli oggetti in base al rapporto valore/peso (*value-to-weight ratio*). Per ottimizzare le prestazioni, l'insieme degli oggetti viene ordinato una sola volta all'inizio del processo e memorizzato nella variabile esterna `sortedItems`. Questo approccio evita di riordinare gli oggetti a ogni esecuzione, mantenendo la complessità computazionale bassa.

A ogni passo, l'algoritmo:

1. Seleziona l'oggetto con il rapporto valore/peso più alto (in testa alla lista `sortedItems`)
2. Verifica se l'oggetto può essere inserito nello zaino senza superarne la capacità
3. Se possibile, inserisce l'oggetto e lo rimuove dalla lista `sortedItems`
4. Ripete il processo fino a esaurimento della capacità residua o degli oggetti disponibili

```
1 executeGreedy() {
2   if (!this.sortedItems.length) return false;
3
4   const bestItem = this.sortedItems[0];
5   const { itemData } = bestItem;
6   const { value, weight } = itemData;
7
8   const canAdd = this.backpack.currentWeight + weight
9   <= this.backpack.maxWeight;
10  if (canAdd) {
11    const inserted = this.backpack.insertItem(
12      bestItem, this.container);
13    if (inserted) {
14      this.updateDisplay();
15    }
16  }
```

```

13         this.sortedItems.shift();
14         return true;
15     }
16 }
17 return false;
18 }

```

Codice 4.11: Esecuzione dell'algoritmo Greedy

La complessità computazionale è dominata dalla fase di ordinamento iniziale ($\mathcal{O}(n \log n)$), mentre la fase di selezione ha complessità $\mathcal{O}(n)$. Questo rende l'algoritmo particolarmente adatto a contesti interattivi che richiedono reattività. Tuttavia, essendo un metodo greedy, non garantisce una soluzione ottimale: effettua scelte localmente migliori che non necessariamente portano al massimo valore complessivo.

4.4.2 Algoritmo di Programmazione Dinamica

L'algoritmo basato sulla programmazione dinamica implementa una soluzione esatta al problema dello zaino. Considera tutte le combinazioni possibili di oggetti, utilizzando una tabella bidimensionale $dp[i][w]$, in cui ogni cella rappresenta il massimo valore ottenibile considerando i primi i oggetti e una capacità residua pari a w . La struttura `dpSolution`, dichiarata esternamente, memorizza la tabella e la capacità residua, consentendo l'applicazione incrementale della soluzione nel gioco.

```

1  executeDP() {
2      if (!this.items.length) return false;
3
4      const capacity = this.backpack.maxWeight - this.
backpack.currentWeight;
5      if (capacity <= 0) return false;
6
7      // Se dpSolution è già calcolata, usa quella
altrimenti:
8      if (!this.dpSolution) {
9          // Calcola la soluzione DP una sola volta
10         const n = this.items.length;
11         const dp = Array.from({ length: n + 1 }, ()
=> Array(capacity + 1).fill(0));
12
13         for (let i = 1; i <= n; i++) {
14             const { value, weight } = this.items[i -
1].itemData;

```

```
15         for (let w = 0; w <= capacity; w++) {
16             if (weight <= w) {
17                 dp[i][w] = Math.max(dp[i - 1][w],
dp[i - 1][w - weight] + value);
18             } else {
19                 dp[i][w] = dp[i - 1][w];
20             }
21         }
22     }
23
24     // Memorizza la soluzione
25     this.dpSolution = { dp, capacity };
26 }
27
28 return this.applyDPSolution();
29 }
```

Codice 4.12: Esecuzione dell'algoritmo di Programmazione Dinamica

Una volta costruita la tabella dinamica, l'algoritmo ricostruisce la soluzione estraendo il primo oggetto appartenente al sottoinsieme ottimale e tentando di inserirlo nel contenitore. Se l'inserimento ha successo, l'oggetto viene rimosso dalla lista e la soluzione viene invalidata per essere ricalcolata al passo successivo.

```
1     applyDPSolution() {
2         if (!this.dpSolution) return false;
3
4         const { dp, capacity } = this.dpSolution;
5         const n = this.items.length;
6         let w = capacity;
7         let chosenIndex = -1;
8
9         for (let i = n; i > 0; i--) {
10             if (dp[i][w] !== dp[i - 1][w]) {
11                 chosenIndex = i - 1;
12                 break;
13             }
14         }
15
16         if (chosenIndex === -1) return false;
17
18         const bestItem = this.items[chosenIndex];
19         const inserted = this.backpack.insertItem(
bestItem, this.container);
```

```
20
21     if (inserted) {
22         this.updateDisplay();
23         this.items.splice(chosenIndex, 1);
24         this.dpSolution = null; // Invalida la
soluzione dopo l'uso
25         return true;
26     }
27     return false;
28 }
```

Codice 4.13: Applicazione della soluzione ottima

L'approccio dinamico presenta una complessità temporale e spaziale pari a $\mathcal{O}(n \cdot C)$, dove n è il numero di oggetti e C la capacità massima dello zaino e sebbene più oneroso in termini computazionali, garantisce sempre la soluzione ottima.

4.4.3 Confronto tra gli Approcci

Il confronto tra i due algoritmi è stato reso esplicito all'interno del gioco, attraverso una sfida diretta tra utente e algoritmo. I risultati mettono in luce quanto segue:

- L'approccio greedy si distingue per la rapidità e semplicità, risultando ideale per contesti interattivi e dimostrativi.
- La programmazione dinamica, pur essendo più complessa e meno reattiva, produce sempre la soluzione ottimale.
- Il confronto tra le due strategie stimola la riflessione critica dello studente sui concetti di efficienza, accuratezza e compromesso computazionale.

L'implementazione di entrambi gli approcci, con le rispettive ottimizzazioni (`sortedItems` per Greedy e `dpSolution` per DP), offre una base solida per comprendere i principi fondamentali dell'ottimizzazione combinatoria e della progettazione di algoritmi.

4.5 Test e Validazione

La fase di test e validazione ha avuto l'obiettivo di verificare il corretto funzionamento dell'applicazione sviluppata e la sua efficacia come strumento di-

dattico per l'introduzione al pensiero computazionale attraverso il problema dello zaino.

Il processo di validazione si è concentrato su tre aspetti principali:

- **Correttezza funzionale:** assicurare che tutte le funzionalità principali — tra cui selezione degli oggetti, inserimento e rimozione dallo zaino, esecuzione dell'algoritmo, gestione dei turni, calcolo dei punteggi — siano operative, coerenti e prive di errori logici o comportamentali.
- **Usabilità:** valutare l'esperienza utente in termini di chiarezza dell'interfaccia, comprensibilità dei messaggi e facilità di interazione con i componenti del gioco. Sono stati osservati aspetti quali la disposizione degli elementi grafici, il feedback fornito all'utente e la reattività del sistema.
- **Efficacia didattica:** misurare la capacità dell'applicazione di supportare l'apprendimento dei concetti legati all'ottimizzazione, alla rappresentazione algoritmica e alla riflessione strategica. In particolare, si è verificato se la struttura a turni e il confronto diretto con l'algoritmo promuovono la comprensione attiva da parte dello studente.

Per raggiungere questi obiettivi sono state definite una serie di strategie di test che includono scenari d'uso significativi e simulazioni delle principali interazioni possibili nel gioco.

4.5.1 Strategia di test

La strategia di test adottata per la validazione dell'applicativo si è basata principalmente su test manuali, con l'obiettivo di verificare il corretto funzionamento delle funzionalità principali, la coerenza dell'interazione con l'utente e l'efficacia comunicativa dell'interfaccia. I test sono stati condotti in ambiente desktop, utilizzando i browser Google Chrome e Mozilla Firefox, con risoluzione standard (1920x1080) e configurazioni di sistema comuni.

La classificazione seguente distingue i test condotti secondo la loro tipologia:

Test funzionali manuali

Questi test sono stati eseguiti per verificare che ogni componente dell'applicativo svolgesse correttamente il proprio compito in condizioni normali.

- **Creazione degli oggetti:** Verifica della corretta generazione casuale di oggetti con valori e pesi differenti, coerentemente rappresentati da barre (proporzionali al peso) e monete gialle (proporzionali al valore).
- **Inserimento nello zaino:** Controllo che un oggetto selezionato venga correttamente posizionato nella griglia dello zaino, rispettando la capacità residua e la disposizione in celle consecutive.
- **Rimozione dallo zaino:** Verifica del funzionamento del pulsante “Rimuovi”, accertandosi che l’oggetto venga eliminato e che gli oggetti successivi si spostino correttamente a sinistra nella griglia.
- **Gestione del turno:** Verifica dell’alternanza tra mossa dell’utente e risposta dell’algoritmo, con l’attivazione e disattivazione corretta dei controlli.
- **Calcolo del punteggio:** Controllo della corretta somma dei valori degli oggetti inseriti e visualizzazione coerente del punteggio utente e algoritmo.
- **Messaggio di fine partita:** Verifica della visualizzazione del messaggio conclusivo con il risultato (vittoria, sconfitta o pareggio).

Test di interazione utente

Questi test si sono concentrati sull’esperienza d’uso e sull’interfaccia, per valutare la chiarezza del flusso operativo e la reattività dell’applicativo alle azioni dell’utente.

- **Selezione degli oggetti:** Verifica dell’effetto visivo al click su un oggetto e del comportamento esclusivo della selezione (solo un oggetto selezionabile alla volta e basta un clic su una sua cella).
- **Feedback visivo:** Controllo che lo stato corrente dell’interfaccia sia comunicato all’utente attraverso elementi visivi chiari (colorazione degli item, messaggi testuali, disattivazione pulsanti).
- **Messaggi e stato del gioco:** Verifica della coerenza tra lo stato interno del gioco (es. turno attivo, algoritmo in esecuzione) e il feedback visivo (es. messaggi informativi, pulsanti disabilitati, animazioni).
- **Blocco e sblocco dell’interfaccia:** Controllo che i pulsanti vengano abilitati/disabilitati correttamente in base al turno o allo stato di gioco, evitando input indesiderati.

Test di compatibilità e stabilità

Pur trattandosi di un progetto prototipale, sono stati eseguiti test base di compatibilità e stabilità, per assicurare un'esperienza coerente su più piattaforme.

- **Compatibilità multi-browser:** L'applicativo è stato testato su Chrome e Firefox per assicurare il corretto rendering dell'interfaccia, delle animazioni e delle interazioni.
- **Stabilità del sistema:** L'applicazione è stata testata su più sessioni consecutive (con reset completo del gioco), per verificare l'assenza di perdite di stato, errori JavaScript o degrado delle performance.

Criteri di successo

Per ogni scenario di test, il criterio di superamento era la completa coerenza tra l'azione eseguita e l'effetto previsto (funzionale o visivo), senza generazione di errori in console, blocchi dell'interfaccia o stati incoerenti.

Tutti i test documentati sono stati superati correttamente dopo una fase iterativa di debug, rendendo l'applicativo stabile e coerente rispetto agli obiettivi iniziali.

4.5.2 Analisi dei risultati

L'analisi dei risultati ottenuti dalla fase di test e validazione evidenzia la maturità raggiunta dall'applicativo in termini di stabilità, correttezza funzionale e qualità dell'esperienza utente. In questa sezione si riportano le principali osservazioni relative agli esiti dei test condotti.

Esito dei test

Tutti i test funzionali manuali, i test di interazione utente e i test di compatibilità descritti nella sezione precedente sono stati completati con esito positivo. L'applicativo si è comportato in modo stabile in ogni scenario considerato, senza errori in console JavaScript, malfunzionamenti dell'interfaccia o comportamenti inattesi.

Stabilità e correttezza

L'applicativo ha dimostrato una buona stabilità in fase di utilizzo, anche durante sessioni ripetute con numerosi reset e inserimenti consecutivi. Le operazioni di creazione oggetti, selezione, inserimento nello zaino, rimozione

e calcolo del punteggio sono risultate corrette e coerenti con le specifiche progettuali. La gestione dei turni tra utente e algoritmo si è rivelata affidabile, mantenendo l'alternanza prevista e impedendo interferenze tra le azioni dei due "giocatori".

Coerenza modello–vista

Durante l'intero ciclo di sviluppo, particolare attenzione è stata posta alla sincronizzazione tra il modello dei dati (oggetti, zaino, stato del gioco) e la vista (interfaccia DOM). I test hanno confermato che le modifiche allo stato interno dell'applicativo sono correttamente riflesse a livello visivo, senza discrepanze tra ciò che è rappresentato sullo schermo e lo stato effettivo del gioco.

Esempi di coerenza modello–vista includono:

- aggiornamento dinamico delle celle dello zaino al momento dell'inserimento o rimozione degli oggetti;
- disattivazione dei pulsanti di azione in base al turno o allo stato del gioco;
- feedback visivo corrispondente alla selezione degli oggetti e alla variazione del punteggio.

Questa coerenza è stata ottenuta anche grazie alla riorganizzazione del codice secondo un'architettura di tipo *Model–View–Controller*, che ha favorito una gestione più chiara e modulare delle entità e dei flussi di interazione.

Esperienza d'uso e facilità di interazione

L'interfaccia utente si è rivelata efficace nel guidare l'utente durante l'interazione con il gioco. Gli elementi interattivi sono stati percepiti come chiari e accessibili, grazie anche all'uso di colori distintivi (monete gialle), pulsanti ben visibili e messaggi informativi nei momenti chiave della partita.

L'esperienza d'uso è risultata intuitiva anche per utenti non esperti, rispecchiando positivamente l'obiettivo didattico del progetto. La semplicità dell'interfaccia, unita al feedback visivo coerente, ha reso l'apprendimento del concetto di algoritmo e ottimizzazione accessibile e coinvolgente.

I risultati della fase di test dimostrano che l'applicativo è pronto per un utilizzo in contesto educativo prototipale. Le funzionalità sono state validate positivamente e l'esperienza complessiva soddisfa i criteri di efficacia, chiarezza e stabilità, in linea con gli obiettivi definiti nel progetto.

Capitolo 5

Risultati

 UNIVERSITÀ
DI PARMA

Corso di Laurea in Informatica

DIPARTIMENTO SCIENZE MATEMATICHE, FISICHE E INFORMATICHE

SFIDA L'ALGORITMO

Algoritmo

valore: 0
Peso: 0 / 20

Avvia Nuova Partita

Scegli l'algoritmo da sfidare:

Greedy

Inserisci Item

Rimuovi Item

Fine Lavoro

Termina Partita

Utente

valore: 0
Peso: 0 / 20

5.1 Nuova partita

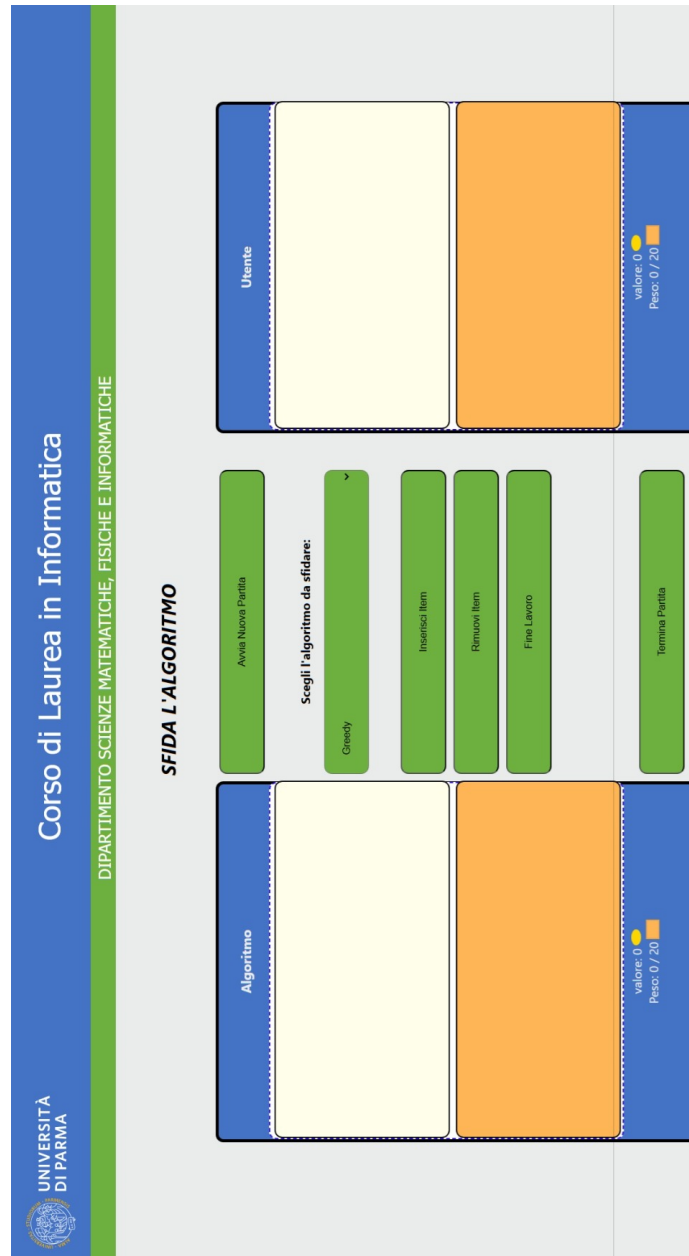


Figura 5.1: Avvio della pagina HTML

5.2 Sfida l'algoritmo

Nel caso dell'algoritmo Greedy:

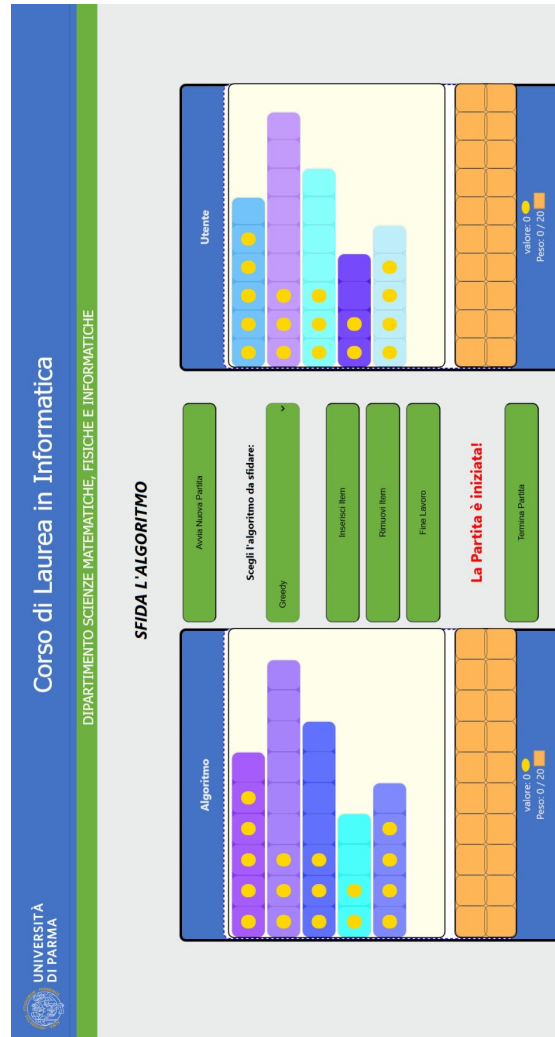


Figura 5.2: Avvio di una nuova partita

Dopo il clic sul pulsante **Avvia Nuova Partita**, gli oggetti vengono generati automaticamente e un messaggio d'inizio della partita viene inviato all'utente.

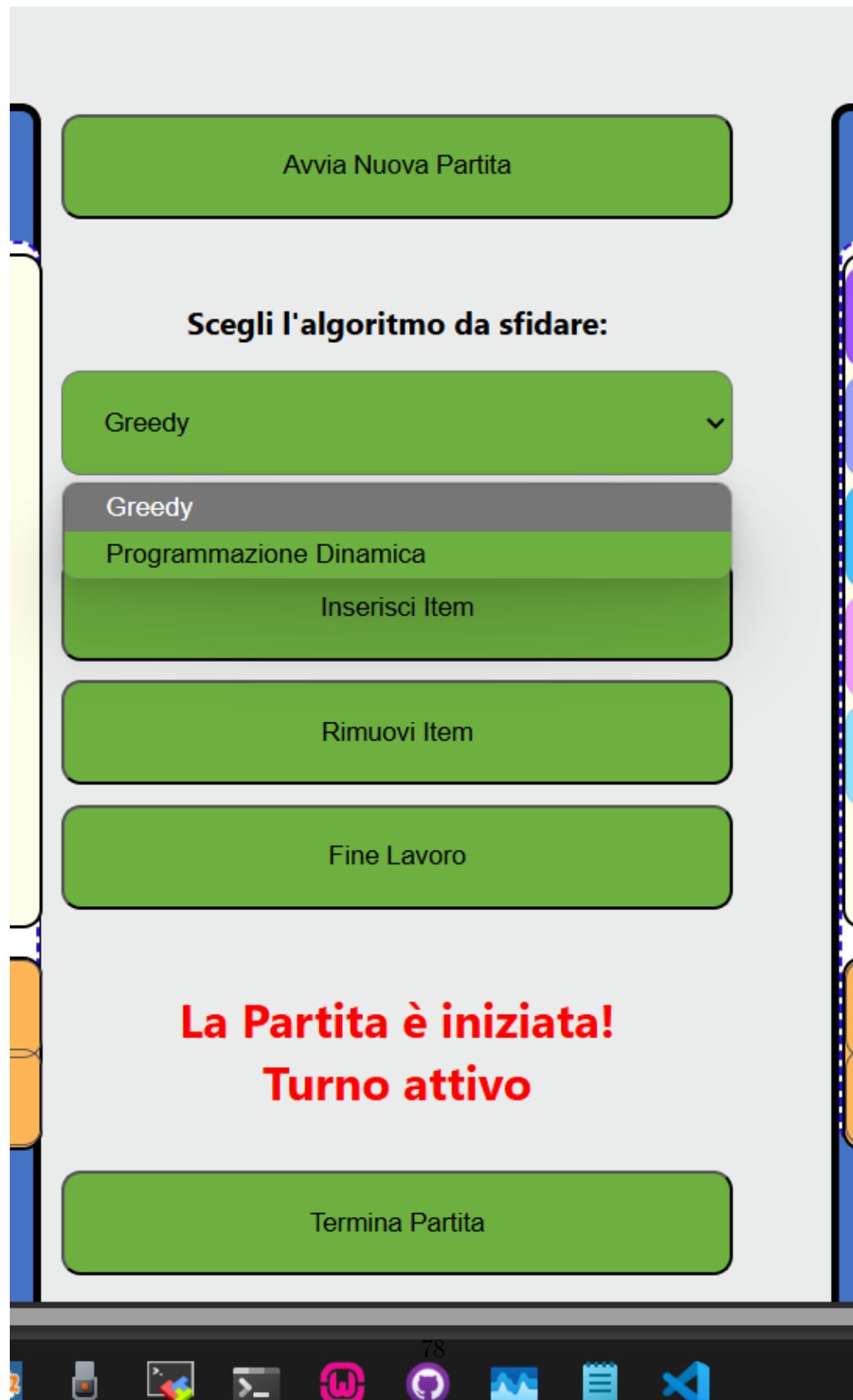


Figura 5.3: Area di controllo

Infatti, nella zona di controllo è possibile vedere diversi pulsanti, tra cui i pulsanti d'interazione, la lista degli algoritmi disponibili per la sfida, e la zona per i messaggi.

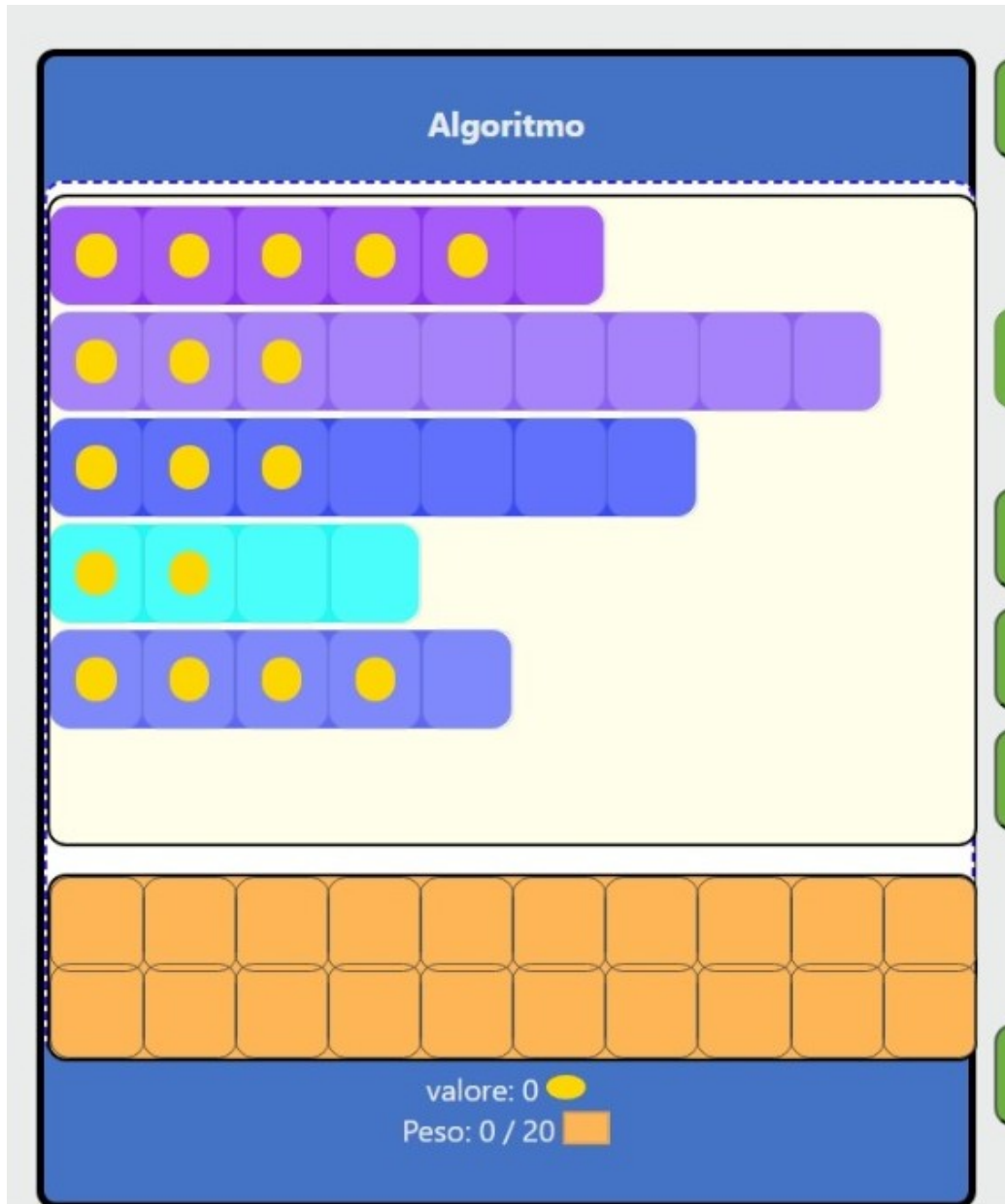


Figura 5.4: Sezione del giocatore (algoritmo)

Oltre a questo, si può osservare chiaramente: la formazione dello zaino a 20 celle, un set di 6 oggetti con caratteristiche differenti (valore, peso, colore).

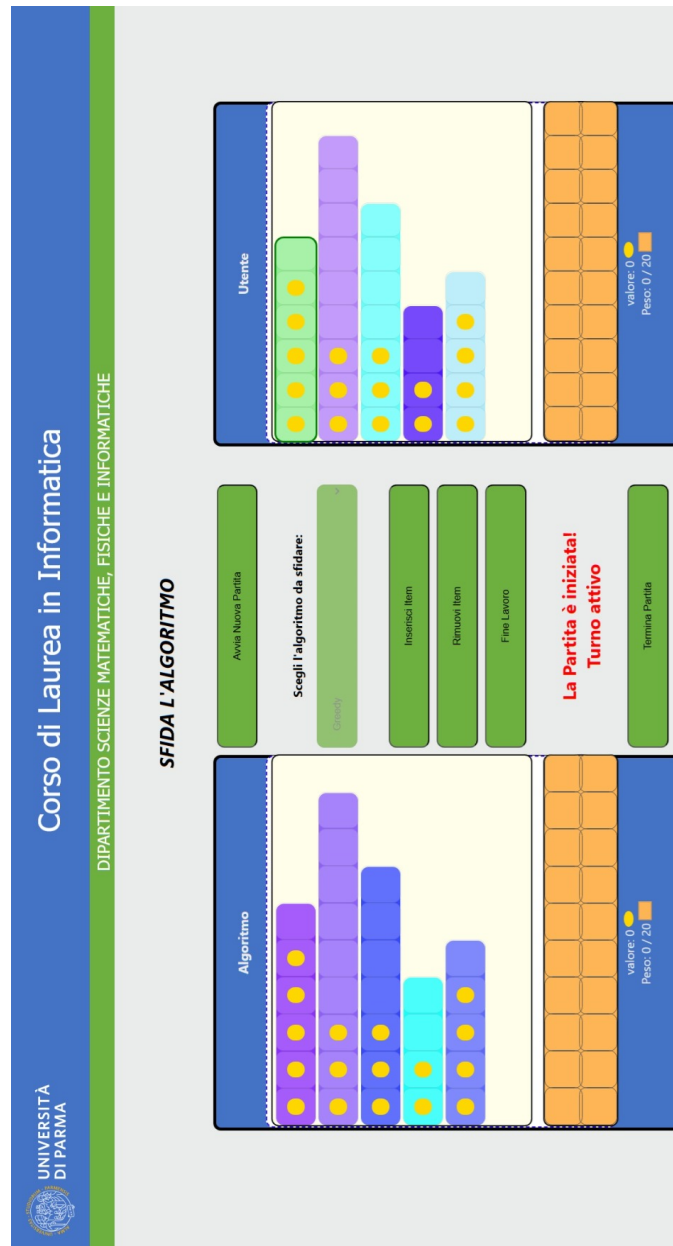


Figura 5.5: Primo turno dell'utente: selezione dell'oggetto

Dopo la prima selezione di un oggetto , l'algoritmo da sfidare viene attivato automaticamente e il menu di scelta viene disattivato.

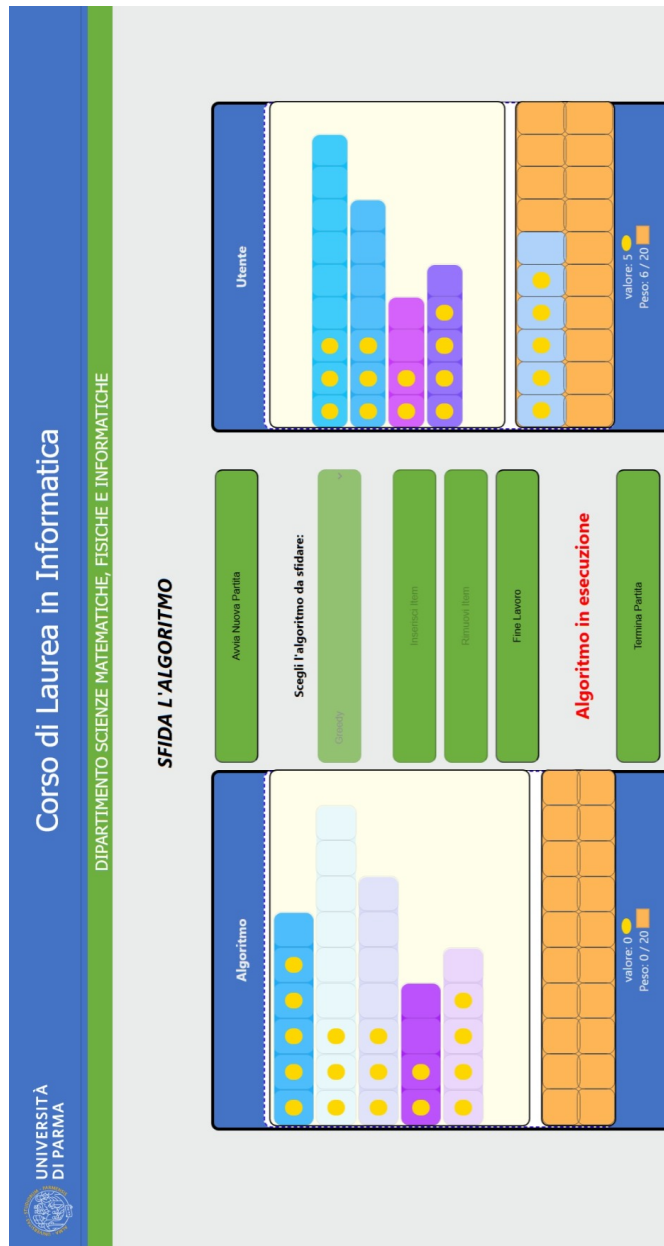


Figura 5.6: Primo turno dell'utente: inserimento dell'oggetto

Poi una volta l'inserimento da parte dell'utente fatta, i pulsanti di interazione vengono disattivati, simulando il passaggio di turno all'algoritmo.

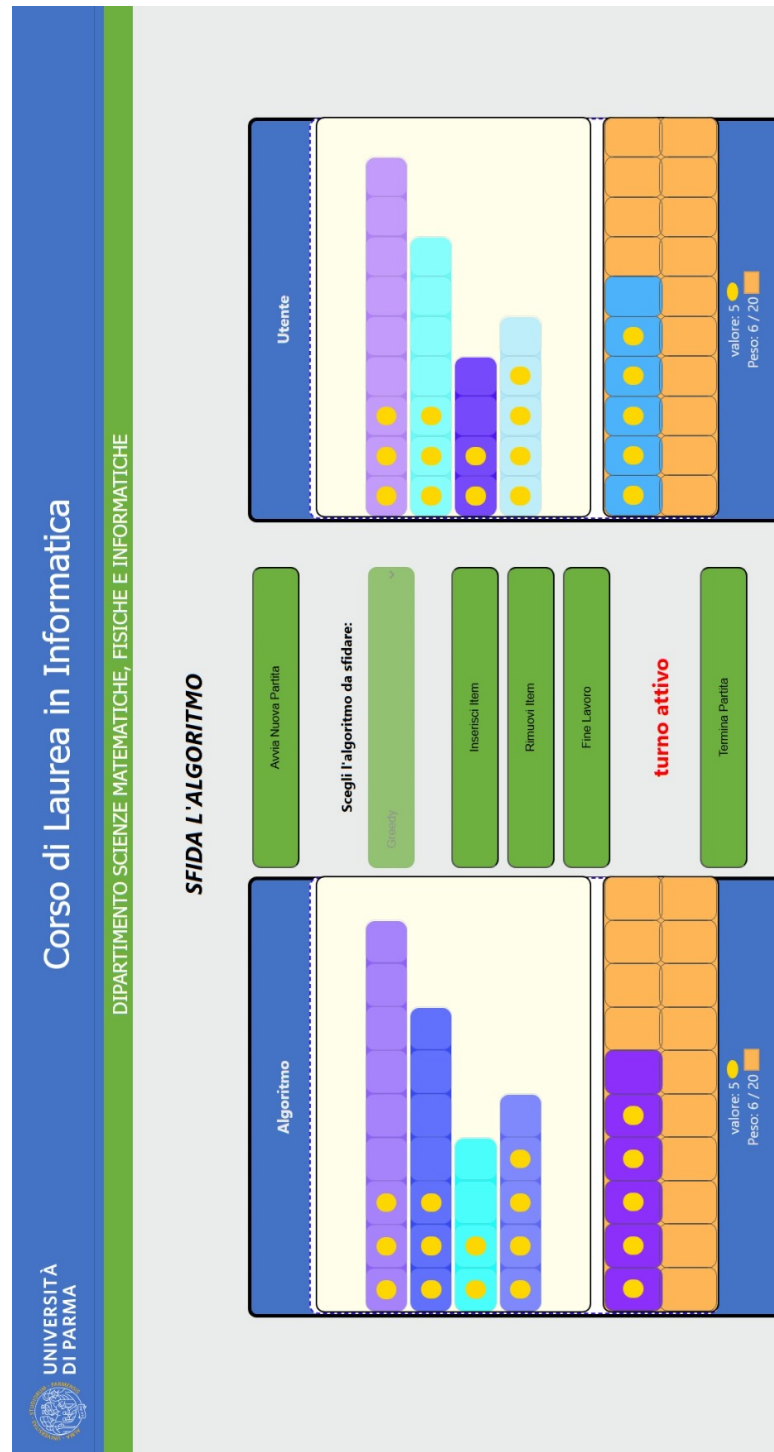


Figura 5.7: Prima mossa dell'algoritmo

5.2.1 Gestione del peso

Nel caso in cui lo spazio residuo dello zaino non sia sufficiente a contenere l'oggetto scelto dall'utente, si presentano due possibili alternative:

Caso 1: Cambiare l'oggetto da inserire

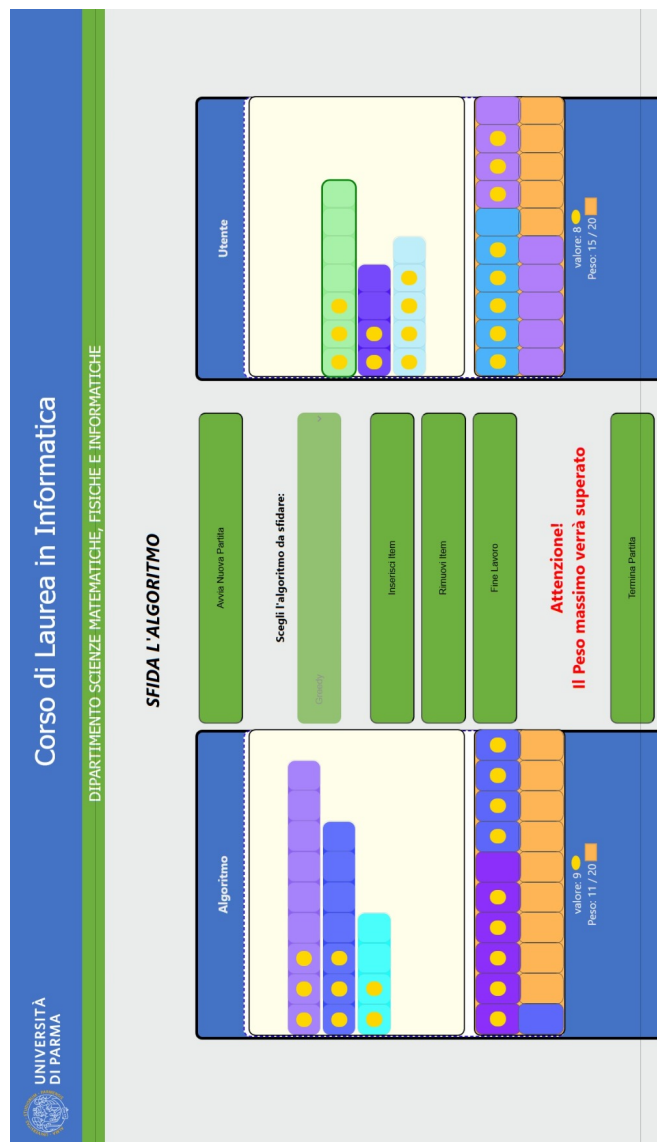


Figura 5.8: Situazione di peso superato: caso 1

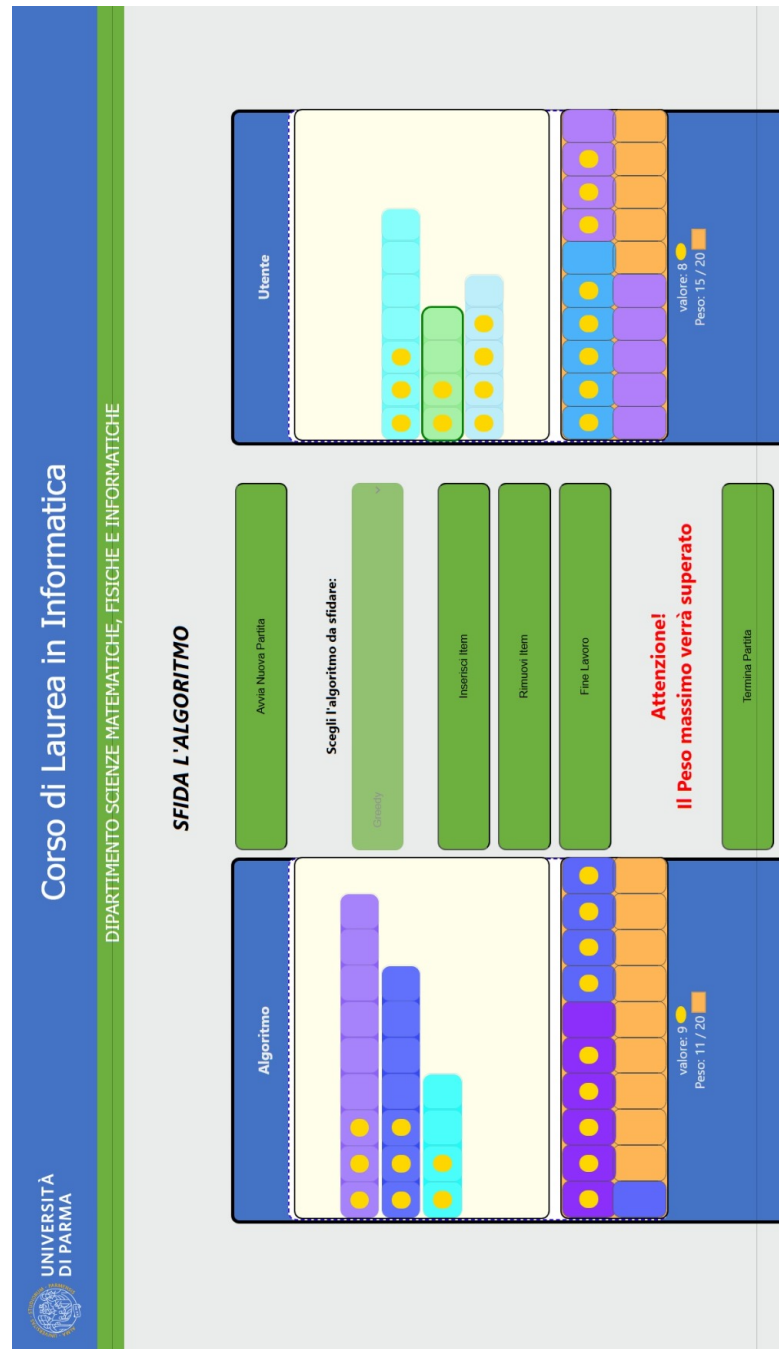


Figura 5.9: Cambio di item (nuova selezione)

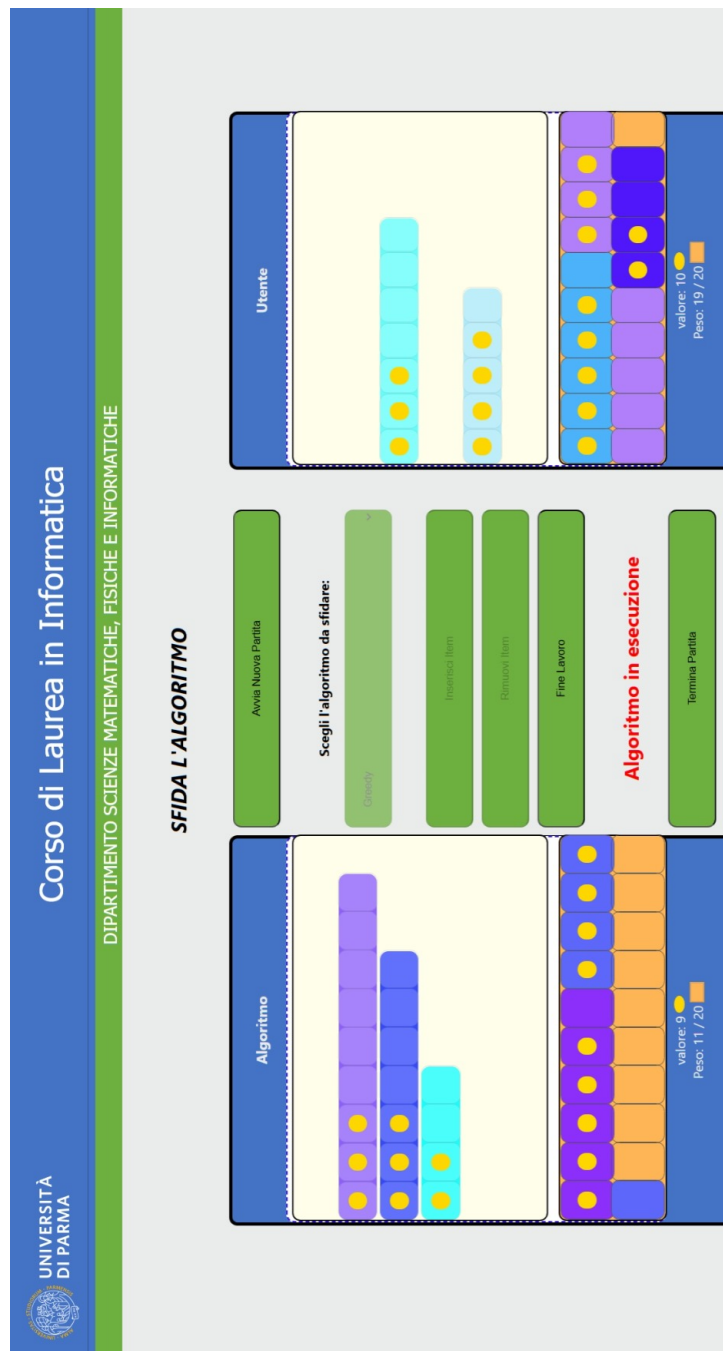


Figura 5.10: Nuovo inserimento riuscito

Caso 2: Liberare spazio nello zaino

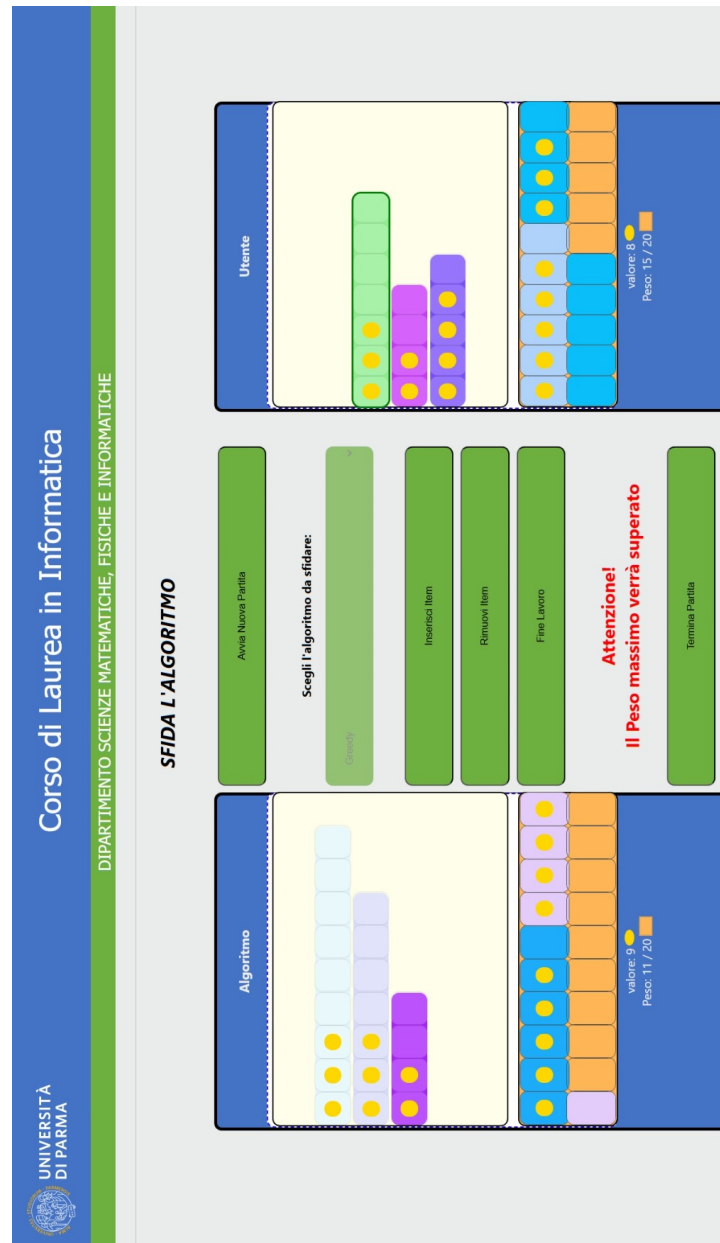


Figura 5.11: Situazione di peso superato: caso 2



Figura 5.12: Selezione e rimozione di un oggetto dallo zaino

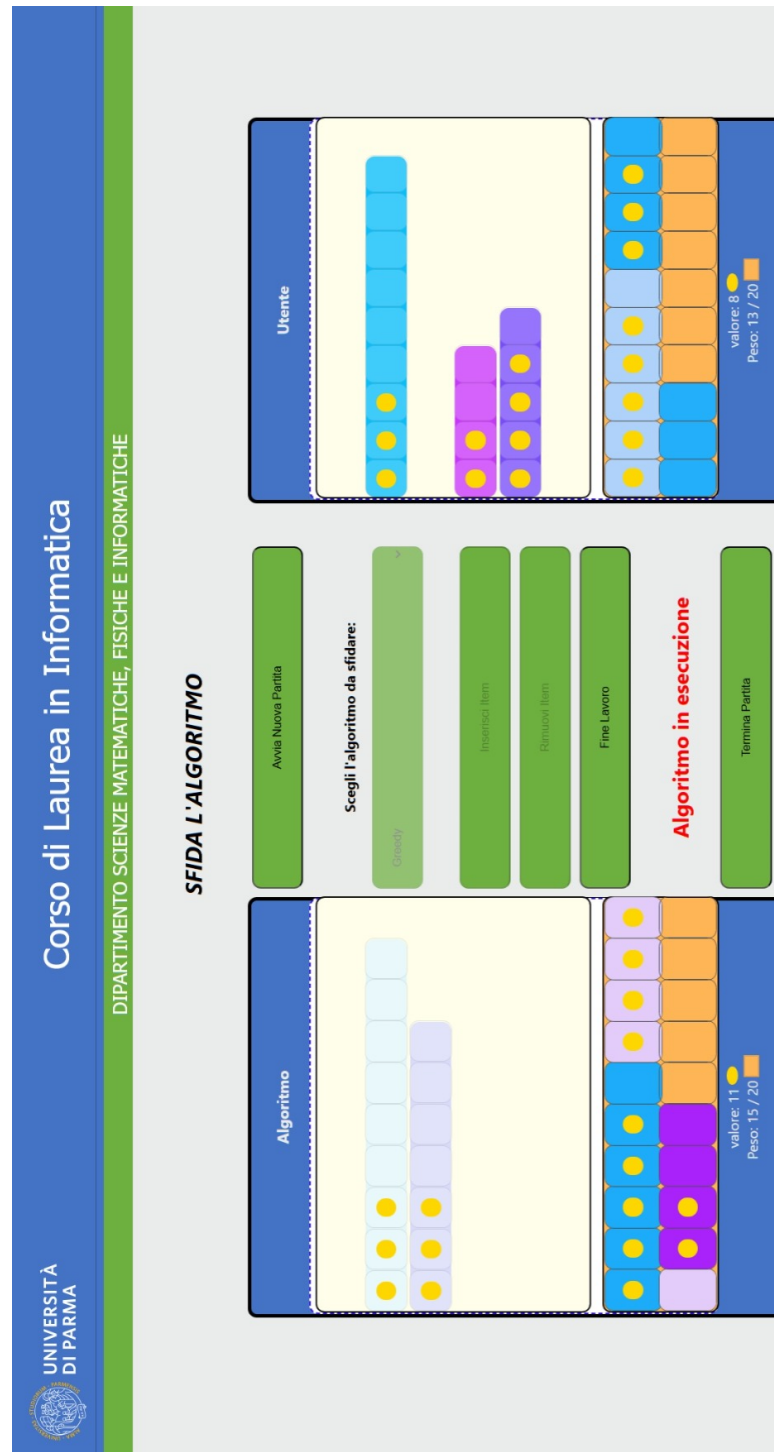


Figura 5.13: Nuovo inserimento riuscito

5.3 Fine della partita e confronto dei risultati

Partendo da una serie di tre confronti diversi tra un utente e un algoritmo , si possono ottenere risultati differenti in base alle loro scelte.

5.3.1 Fine della partita: Utente vs Greedy

In base alla successione di turni tra utente e algoritmo di Greedy durante la partita precedente, si passa alla valutazione finale.



Figura 5.14: Fine della partita

5.3.2 Caso dell'algoritmo di Programmazione Dinamica

Caso 1: Vittoria dell'algoritmo DP

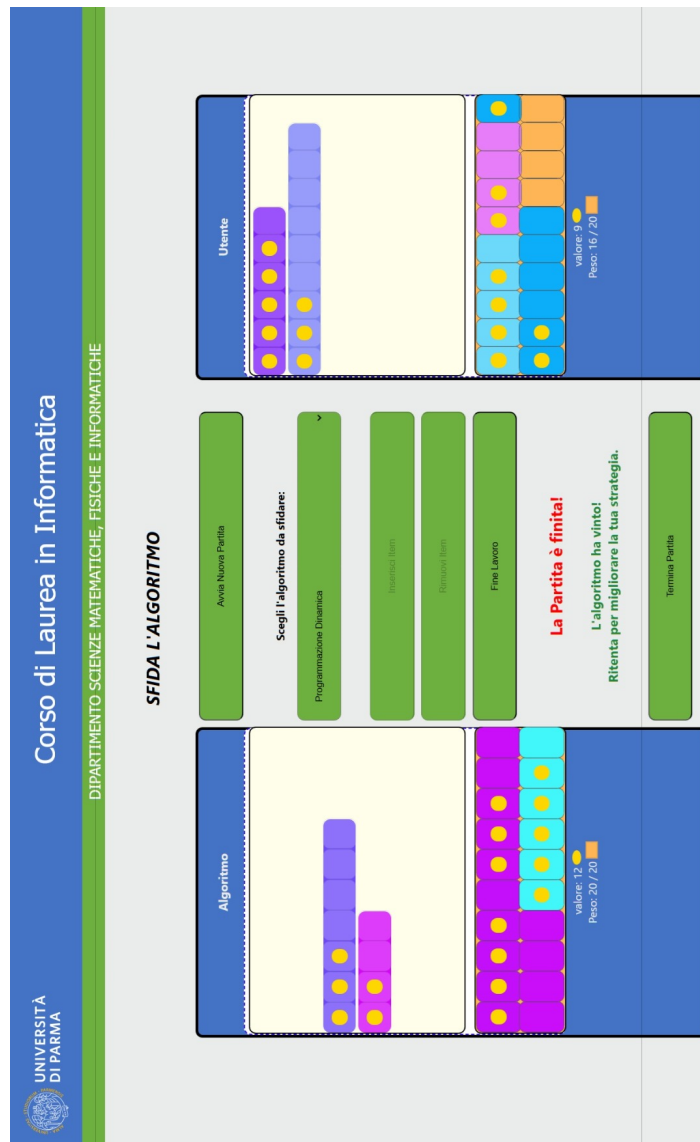


Figura 5.15: Fine della partita: vittoria dell'algoritmo

In questo caso di vittoria dell'algoritmo DP, ha ottenuto i seguenti risultati: peso = 20/20, valore = 12; mentre l'utente ha totalizzato: peso = 16/20, valore = 9.

Caso 2: Vittoria dell'utente



Figura 5.16: Fine della partita: vittoria dell'utente

In questo caso, l'algoritmo DP ha ottenuto: peso = 20/20, valore = 12; mentre l'utente ha totalizzato: peso = 18/20, valore = 12. Quindi entrambi hanno ottenuto lo Stesso valore ma l'utente avendo utilizzato meno spazio ha vinto.

Caso 3: Pareggio

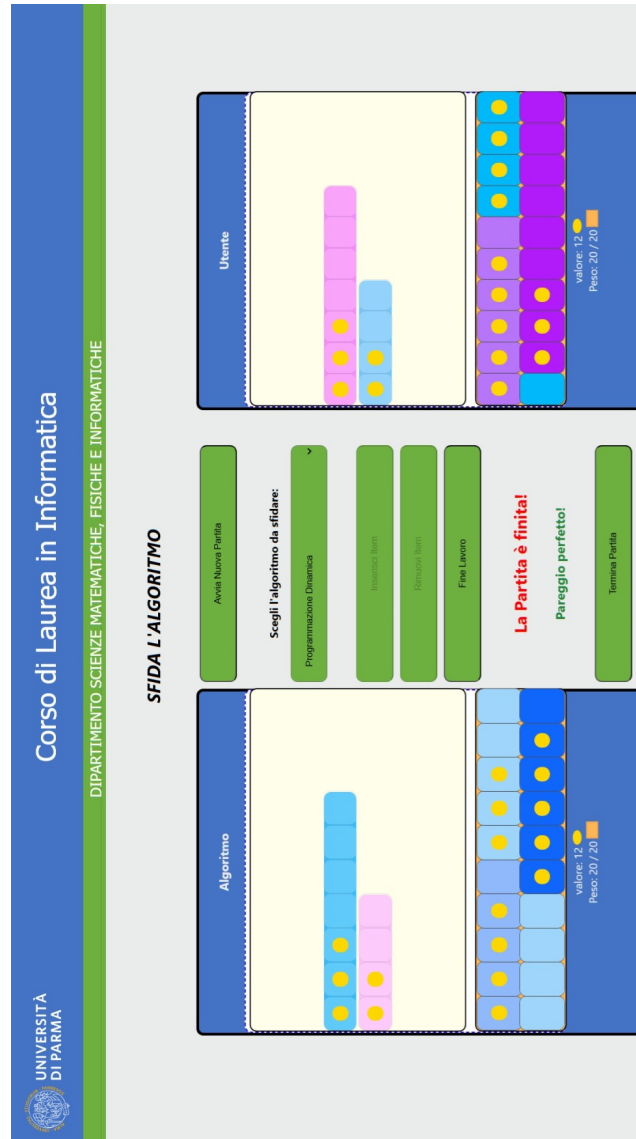


Figura 5.17: Fine della partita: pareggio

In questo caso si è verificato un pareggio.

In base al confronto tra i risultati ottenuti, si può osservare che l'algoritmo Greedy non restituisce sempre la soluzione ottimale rispetto alla Programmazione Dinamica. Tuttavia, quest'ultima tende a massimizzare il valore totale senza necessariamente risparmiare peso.

Conclusione

L'elaborato presentato si è posto l'obiettivo di esplorare e valorizzare il potenziale delle metodologie interattive per l'insegnamento del pensiero computazionale, attraverso lo sviluppo di un'applicazione didattica basata sul problema dello zaino. L'approccio adottato, fondato sul *learning by doing* e sull'impiego di elementi ludici, ha permesso di coniugare l'aspetto formativo con il coinvolgimento attivo degli studenti, offrendo un ambiente digitale accessibile, stimolante e visivamente efficace per l'apprendimento di uno dei problemi algoritmici più rappresentativi della programmazione.

Nel corso del progetto è stata realizzata un'applicazione interattiva, sviluppata interamente con tecnologie web di base (HTML, CSS e JavaScript), pensata per guidare l'utente attraverso un processo esplorativo e visivo di risoluzione del problema. L'architettura dell'applicazione si fonda su una rappresentazione grafica degli oggetti, su un'interfaccia utente intuitiva e su una logica ben organizzata in componenti di dati, interazione e vista, ispirata al modello MVC. Tale struttura ha reso il codice modulare e facilmente estendibile, facilitando eventuali sviluppi futuri.

Di più, i test condotti nella fase di validazione hanno evidenziato la solidità dell'applicazione sotto il profilo funzionale e una buona coerenza tra modello e vista. Gli utenti coinvolti nelle simulazioni hanno mostrato di apprezzare l'interattività, l'immediatezza del feedback e la possibilità di confrontare il proprio ragionamento con quello dell'algoritmo. La componente ludica ha reso l'esperienza di apprendimento non solo efficace, ma anche motivante e gratificante.

Sotto il profilo didattico, il progetto ha dimostrato come l'utilizzo di simulazioni visuali e interazioni concrete possa facilitare la comprensione di concetti complessi legati all'ottimizzazione combinatoria, quali la selezione di oggetti sotto vincoli e la massimizzazione del valore totale. La possibilità di manipolare direttamente gli oggetti e osservare le conseguenze delle proprie scelte promuove un approccio attivo al *problem solving*, stimolando le capacità logiche e la riflessione strategica.

Sebbene i risultati ottenuti siano soddisfacenti, il progetto presenta nu-

merosi margini di miglioramento. Tra le estensioni possibili si annoverano: l'introduzione di livelli di difficoltà progressiva, l'inclusione di varianti del problema (come lo zaino frazionario o il multi-knapsack), il supporto per la rotazione e la manipolazione avanzata di oggetti in 2D o 3D, una modalità multiplayer per il confronto diretto tra utenti, la registrazione dei punteggi e delle statistiche delle partite, nonché la visualizzazione dettagliata dell'esecuzione degli algoritmi. Ulteriori sviluppi potrebbero includere l'integrazione di metodi risolutivi più avanzati, come l'algoritmo *branch and bound*, per un confronto più approfondito tra strategie euristiche e soluzioni ottimali.

In conclusione, questa tesi ha voluto dimostrare come un'attività di progettazione software possa configurarsi anche come un percorso di ricerca educativa, in cui la tecnologia non rappresenta un fine, bensì un mezzo per abilitare nuove forme di apprendimento. Il successo dell'esperienza proposta conferma il valore del *game-based learning* e ne auspica una sempre più ampia diffusione nella didattica dell'informatica, a sostegno di un'educazione digitale più creativa, inclusiva ed efficace. L'applicazione realizzata rappresenta dunque un primo passo concreto verso lo sviluppo di strumenti digitali orientati all'educazione algoritmica e alla promozione del pensiero computazionale nella scuola.

Bibliografia

- [Bell et al., 1998] Bell, T., Witten, I. H., and Fellows, M. (1998). Computer science unplugged: Enrichment and extension activities for primary-aged children. <https://csunplugged.org>.
- [Cormen et al., 2009] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. MIT Press, 3 edition.
- [Domestrescu et al., 2013] Domestrescu, E., Finocchi, I., and Italiano, G. F. (2013). *Algoritmi e Strutture Dati*. McGraw-Hill Education, Milano.
- [Felker and Purao, 2020] Felker, K. and Purao, S. S. (2020). Gamification as a tool for enhancing learning. *ACM Transactions on Computing Education*, 20(4):1–28.
- [Gadanidis, 2017] Gadanidis, G. (2017). Five key ideas for teaching computational thinking. *Digital Experiences in Mathematics Education*, 3(2):113–123.
- [Google for Education, 2024] Google for Education (2024). Cs first - free computer science curriculum. <https://csfirst.withgoogle.com>.
- [Grover and Pea, 2013] Grover, S. and Pea, R. (2013). Computational thinking in k-12: A review of the state of the field. In *Proceedings of the 2013 Conference on Learning Sciences (ICLS)*, pages 38–41.
- [Martello and Toth, 1990] Martello, S. and Toth, P. (1990). *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons.
- [Papert, 1980] Papert, S. (1980). *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books.
- [Wing, 2006] Wing, J. M. (2006). Computational thinking. *Communications of the ACM*, 49(3):33–35.

Ringraziamenti

Desidero esprimere la mia più sincera gratitudine al Prof. Vincenzo Bonnici, relatore di questa tesi, per avermi guidato con professionalità e disponibilità lungo tutto questo percorso. Il suo supporto è stato fondamentale non solo nella fase di ricerca, ma soprattutto nello sviluppo del progetto, dove le sue indicazioni e i suoi consigli hanno fatto la differenza. La sua passione per l'informatica mi ha ispirato e motivato a dare il massimo in questo lavoro.

Un ringraziamento speciale va a tutti i docenti che ho incontrato durante il mio percorso universitario. Le loro lezioni, la loro disponibilità e i loro consigli sono stati essenziali per la mia crescita personale e professionale, fornendomi le basi e la motivazione necessarie per portare a termine questo percorso.

Vorrei inoltre ringraziare di cuore i miei amici e colleghi di corso, che mi hanno incoraggiato e sostenuto con entusiasmo. Il loro aiuto nello studio, la loro compagnia e la loro capacità di motivarmi nei momenti di difficoltà hanno reso questo cammino più leggero e stimolante. La loro presenza nei periodi più complessi è stata fondamentale per trovare la forza e la determinazione necessarie a raggiungere questo importante traguardo.

Infine, il mio più sentito ringraziamento va alla mia famiglia, in particolare ai miei genitori, che mi hanno sostenuto con affetto e pazienza durante tutto il percorso universitario, e ai miei zii Josué e Christian, che mi hanno aiutato in molti aspetti. La loro vicinanza, il loro supporto e i loro consigli sono stati determinanti per superare le difficoltà e completare con successo questo percorso di studi.

A tutti voi, il mio più sincero grazie.